CS336 Assignment 5 (alignment): Reasoning RL

Version 26.0.0

CS336 Staff

Spring 2026

1 Assignment Overview

In this assignment, you will gain some hands-on experience training a language model to reason and solve downstream tasks.

What you will implement

1. Zero-shot, few-shot, and chain-of-thought prompting.
2. Group Relative Policy Optimization (GRPO), a reinforcement learning algorithm to improve model performance given external rewards.
3. Policy gradient estimation variants, to explore variance reduction and importance weight clipping strategies.

What you will run

1. Measure prompting performance for OLMo-2-0425-1B on GSM8K.
2. Run on-policy GRPO on OLMo-2-0425-1B to improve performance on GSM8K.
3. Run RL variants, including RFT, Dr. GRPO, and MaxRL, to explore algorithmic choices in RL.
4. Run off-policy GRPO, to speed up training and explore various clipping strategies.

What the code looks like

All the assignment code as well as this writeup are available on GitHub at: github.com/stanford-cs336/assignment5-alignment Please `git clone` the repository. If there are any updates, we will notify you and you can `git pull` to get the latest.

1. `cs336_alignment/*`: This is where you'll write your code for assignment 5. Note that there's no code in here (aside from the starter code described below), so you should be able to do whatever you want from scratch.
2. `cs336_alignment/vllm_utils.py`: This file contains code to run a vLLM server that produces generations and can sync weights.
3. `cs336_alignment/drgrpo_grader.py`: This file contains code to grade model outputs for math problems.
4. `cs336_alignment/prompts/*`: For your convenience, we've provided text files with prompts.
5. `tests/*.py`: This directory contains the required GRPO tests and optional alignment/safety supplement tests.

The required tests in `tests/test_grpo.py` invoke the hooks defined in `tests/adapters.py`. You'll implement the adapters to connect your code to the tests. Writing more tests and/or modifying the test code can be helpful for debugging your code, but your implementation is expected to pass the original provided test suite.

1. `README.md`: This file contains some basic instructions on setting up your environment.

How to submit

You will submit the following files to Gradescope:

- `writeup.pdf`: Answer all the written questions. Please typeset your responses.
- `code.zip`: Contains all the code you've written.

Run the script in `test_and_make_submission.sh` to create the `code.zip` file.

2 Introduction

2.1 Background

In the first four assignments of this class, we learned about how to pretrain a base model. We're now ready to learn about post-training: once we have a base model, how do we turn it into a useful tool that can solve downstream tasks?

One component of post-training is *alignment*. Pre-training, due to both its data mixture and training objective, has the effect of imbuing a model with a broad range of knowledge and behaviors. But when we ask the model to solve tasks, we want a specific behavior, that of a helpful and harmless assistant. The process of turning a broad base model into a focused chat model is called "instruction tuning" or "alignment," and we'll cover these techniques in the optional Assignment 5 supplement.

One other component of post-training is *reinforcement learning*. In pre-training, because our goal is to imbue the model with a broad knowledge base, we use a broad set of data (text on the web). In post-training RL, our goal narrows: we want the model to attain high accuracy on a specific task, like solving math problems. This narrower goal differs from pre-training in at least two ways: (a) we no longer have as much data, and (b) our training objective changes from one of "coverage" (having a broad knowledge base) to one of "precision" (producing accurate responses). As a result of these differences, we need a new technique, namely reinforcement learning (RL).

In RL, we are given a dataset of questions, and a scoring function that judges whether a response correctly solves the question. For example, in a coding setting, we might be given a coding problem (e.g. "Write a Python function that reverses a list"), and a scoring function consisting of a set of test cases (e.g. `assert f([0, 1, 2]) == [2, 1, 0]`). Or in a math setting, we might be given a mathematical reasoning word problem (e.g. "If Tom was half John's age four years ago and John is now twenty, how old is Tom now?"), and a scoring function that parses the final answer and checks that it is correct (`12`). Notably, unlike in pretraining, we are not given a dataset of responses to mimic: in the coding example, we are not given a correct Python program that solves the task, and in the mathematical reasoning example, we are not given a correct chain of reasoning that produces the answer. Instead, in RL, we directly take gradient steps on model accuracy as the objective function. At a high level, RL will involve sampling responses from the model, grading them with the scoring function, and then upweighting the ones that are correct.

In this assignment, we'll learn about RL both mathematically and empirically. It turns out that RL is hard, in that it is slow and unstable. Doing science for RL is also hard, in that RL runs have high variance across random seeds, and seemingly small details in implementation matter a lot. This assignment will introduce LLM RL and explore some of its challenges.

2.2 Model and dataset

For this assignment, we will be using OLMo-2-0425-1B, a base model that was pre-trained on OLMo-mix-1124 and mid-trained on Dolmino-mix-1124, with 4 trillion tokens total. OLMo-mix-1124 consists mostly of DCLM-Baseline [J. Li et al., 2024], a dataset we've covered in lecture, while Dolmino-mix-1124 is a more focused data mixture containing roughly 50% DCLM and 50% instruction following, math, code, STEM paper, and wiki data. Other OLMo-2-0425-1B details can be found in the OLMo 2

technical report [T. OLMo et al., 2024]. At this point in the class you should have all the background needed to understand each of their design decisions.

As our downstream task, we will use the GSM8K dataset [K. Cobbe et al., 2021], which is available in the assignment repository at data/gsm8k/train.jsonl and data/gsm8k/test.jsonl, or online here. This dataset contains a relatively easy collection of grade school mathematical reasoning word problems. An example is below:

```
{
  "question": "Natalia sold clips to 48 of her friends in April, and then she sold half as
many clips in May. How many clips did Natalia sell altogether in April and May?",
  "answer": "Natalia sold 48/2 = <<48/2=24>>24 clips in May.\nNatalia sold 48+24 =
<<48+24=72>>72 clips altogether in April and May.\n#### 72"
}
```

During RL, our model will learn to produce reasoning chains for problems of this style, improving its ability to solve math problems.

A note on model and dataset choice: due to resource constraints, we've chosen just one small-scale model-dataset combination to serve as our RL testbed. In particular, we've chosen a small model that's quite capable due to being trained on a very large amount of tokens (4000x the parameter count!), allowing us to see reasonable RL results at small scales on a realistic dataset. The models we trained in this class are unfortunately too weak to solve math problems. If you're curious after completing this assignment, you're also welcome to take the model you trained in this class and run RL on an easier task. Since RL training dynamics are very model- and dataset-dependent, you should keep in mind that the results we see in this assignment may not transfer to other model-dataset combinations.

2.3 Notation

This assignment will have some math in it, so in the following table we provide some of the notation we will use later. Language modeling and reinforcement learning often use different terms for the same object, so the table includes both terms. Feel free to come back to this table if you've forgotten what a piece of notation refers to.

Notation	LM terminology	RL terminology	Meaning
ρ	prompt distribution or dataset	initial state distribution	Distribution over prompts/problems.
x	prompt; question	initial state	A prompt/problem sampled from ρ.
y	response; completion; generation; sample	rollout; trajectory; sampled action sequence	A sampled answer to prompt x.
y_t	token	action	The t-th generated token in y.
$y_{<t}$	prefix		All generated tokens before position t, or $y_1, \ldots, y_{t-1}$.
π_θ	model	policy	A model with parameters θ, which assigns probabilities $\pi_\theta(y \mid x)$ to responses y given prompts x.

$\pi_\theta(y_t \mid x, y_{<t})$	next-token distribution	policy at timestep t	Conditional probability of token y_t given the prompt and previous generated tokens.
$r(y \mid x)$		reward	Scalar score denoting the correctness of a sampled response (0 or 1 in this assignment).
B	number of prompts per batch		Number of prompts per inference batch.
G	generations per prompt	group size	Number of responses sampled for each prompt.
$\text{len}(y)$ or L	response length	horizon	Number of generated tokens in a response.
$A^{(i,j)}$		advantage	Weight assigned to response j for prompt i after baseline and normalization.

3 Prompting

Our first step in applying our pretrained base model to downstream tasks is to prompt it. Base models are pretrained on a wide range of behaviors, and prompting is a lightweight way to shape its behavior toward task-solving modes. Later in the assignment, we'll also see that the prompt choice shapes RL dynamics and exploration.

The most basic way to prompt the model is to provide the question, and sample from its next token distribution to produce the answer; we will call this strategy `question_only`. We will compare this strategy to the `r1_zero` prompt, which includes both the question and instructions for the model to do chain-of-thought reasoning [DeepSeek-AI et al., 2025].

3.1 Using vLLM for inference

To generate responses from the model, we will need an inference engine. Implementing an inference engine is outside the scope of this assignment, so we will use the vLLM inference engine [W. Kwon et al., 2023], which implements a variety of optimizations including fast CUDA kernels, PagedAttention for efficient attention KV caching, and so on. Code to start a vLLM server and produce generations has been provided in `cs336_alignment/vllm_utils.py`, which has the following interface:

```
@dataclass
class VLLMCompletion:
    text: str
    token_ids: list[int]
    finish_reason: str | None


@dataclass
class VLLMServer:
    model_id: str
    gpu: int = 0
    seed: int = 0
    gpu_memory_utilization: float = 0.9
```

```python
    def start(self) -> None: ...

    def generate_completions(
        self,
        prompts: list[str],
        sampling_params: dict,
        batch_size: int | None = None,
    ) -> list[VLLMCompletion]: ...
```

3.2 Zero-shot, few-shot, and chain-of-thought prompting

Unless otherwise specified, for experiments on GSM8K we will use the following prompt from the DeepSeek R1-Zero model [DeepSeek-AI et al., 2025]. We will refer to this as the `r1_zero` prompt:

```
A conversation between User and Assistant. The User asks a question, and the Assistant solves
it. The Assistant first thinks about the reasoning process in the mind and then provides the
User with the answer. The reasoning process is enclosed within <think> </think> and the
answer is enclosed within <answer> </answer> tags, respectively, i.e., <think> reasoning
process here </think> <answer> answer here </answer>.
User: {question}
Assistant: <think>
```

The `r1_zero` prompt is located in the text file `cs336_alignment/prompts/r1_zero.prompt`.

In the prompt, `question` refers to some question that we insert (e.g., `Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did Natalia sell altogether in April and May?`). The expectation is that the model plays the role of the assistant, and starts generating the thinking process (since we have already included an opening `<think>` tag), closes the thinking process with `</think>`, and then generates a final symbolic answer within the answer tags, like `<answer> 4x + 10 </answer>`. The purpose of having the model generate tags like `<answer> </answer>` is so that we can easily parse the model's output and compare it against a ground truth answer, and so that we can stop response generation when we see the closing answer tag `</answer>`.

Another prompting approach, called few-shot prompting, is to prepend a few question-answer pairs before prompting the model with the actual question. A few-shot version of the `r1_zero` prompt looks like the following:

```
A conversation between User and Assistant. The User asks a question, and the Assistant solves
it. The Assistant first thinks about the reasoning process in the mind and then provides the
User with the answer. The reasoning process is enclosed within <think> </think> and the
answer is enclosed within <answer> </answer> tags, respectively, i.e., <think> reasoning
process here </think> <answer> answer here </answer>.
User: {question-1}
Assistant: <think> {reasoning-1} </think> <answer> {answer-1} </answer>
User: {question-2}
Assistant: <think> {reasoning-2} </think> <answer> {answer-2} </answer>
User: {question-3}
Assistant: <think> {reasoning-3} </think> <answer> {answer-3} </answer>
User: {question}
Assistant: <think>
```

Few-shot prompting improves the model's performance by giving it a few examples of the task it is meant to solve. The reported benchmark numbers for open base models are often few-shot: for example, the result reported for GSM8K on OLMo-2-0425-1B's model card uses 8-shot prompting. We've included a 3-shot version of the `r1_zero` prompt in `cs336_alignment/prompts/r1_zero_three_shot_gsm8k.prompt`, with examples taken from the OLMES repository.

Finally, as a baseline, we will include the `question_only` prompt, available at `cs336_alignment/prompts/question_only.prompt`:

```
{question} Please put your final answer within \\boxed{{}}.
```

Note that while the prompt is called `question_only`, we ask for the final answer to be boxed to make it easier for a grader to parse the answer from the response, as discussed in the following section.

3.3 Grading function

Once the model generates a response, we need to check whether it is correct. Our math problems include ground truth answers (e.g. `0.5`), but the model can provide a correct answer in many ways: for example, it can answer `<answer> 1/2 </answer>` or `The answer is 0.5`. So to properly grade model outputs, we need an answer parsing function that takes as input the model's output and a known ground-truth, and returns a boolean indicating whether the model's output is correct.

For our experiments, we will use a fast and fairly accurate answer parser used in recent work on reasoning RL [Z. Liu et al., 2025]. For the `r1_zero` prompts, this reward function is implemented at `cs336_alignment.drgrpo_grader.r1_zero_reward_fn`. The `question_only` prompt does not ask the model to use `<think>` and `<answer>` tags, so it should instead be evaluated with `cs336_alignment.drgrpo_grader.question_only_reward_fn` from the same grader file. Note that the `question_only` parser looks for a final answer in `\boxed{}`. These reward functions return a total reward, along with separate format and answer rewards that indicate whether the response follows the expected format and whether the parsed answer is correct. Some works give partial credit (a non-zero total reward) for responses that are formatted correctly but incorrect. In our experiments, we will not give partial credit, so total reward will just be the answer reward, and we will only use format reward for logging.

Note that these reward functions expect the `ground_truth` argument to contain just the answer. To extract the answer from ground truth responses in the GSM8K dataset, which contain `{rationale} #### {answer}`, you should split on `####` and strip whitespace.

3.4 Experiments

We're now ready to evaluate the base model's prompting performance.

Generation hyperparameters. When generating responses, we'll sample with temperature 1.0, top-p 1.0, max generation length 512. The `r1_zero` prompts ask the model to end its answer with the string `</answer>`, so when using those prompts we can direct vLLM to stop when the model outputs this string:

```
# Based on Dr. GRPO: stop when the model completes its answer
# https://github.com/sail-sg/understand-r1-zero/blob/
#   c18804602b85da9e88b4aeeb6c43e2f08c594fbc/train_zero_math.py#L167
sampling_params['stop'] = ["</answer>"]
sampling_params['include_stop_str_in_output'] = True
```

Only use this stop string for the `r1_zero` and `r1_zero_three_shot` prompts, not for the `question_only` prompt.

Problem (`prompting_baselines`): Run OLMo-2-0425-1B on GSM8K (5 points)

(a) Write a script to evaluate OLMo-2-0425-1B performance on GSM8K with zero-shot `question_only`, zero-shot `r1_zero`, and few-shot `r1_zero_three_shot` prompts.

Then, run your script and observe the outputs. For each prompt, how many model generations fall into each of the following categories: (1) correct with both format and correctness reward 1, (2) format reward 1 and correctness reward 0, (3) format reward 0 and correctness reward 0? Observing at least ten examples of category 2, how many model outputs are actually correct but just not parsed properly? What about category 3?

Deliverable: A few sentences of commentary, the evaluation metrics, and a few examples of prompts and responses.

(b) Observing the model outputs, characterize the model's behavior with each prompt. For example, if we want the model to answer the question, is it enough to just provide the question, or does the model exhibit other behaviors besides just answering the question? How do the zero-shot `r1_zero` and few-shot `r1_zero_three_shot` prompts shape the model's behavior?

Deliverable: A few sentences of commentary with supporting examples.

4 Group Relative Policy Optimization

Now that we've measured the model's performance with just prompting, our next step will be to improve this performance via training. Specifically, we want to optimize the model's accuracy, or equivalently, its expected reward:

$$J_\theta = E_{x\sim\rho} E_{y\sim\pi_\theta(y \mid x)}[r(y \mid x)], \tag{1}$$

where ρ is our task distribution over prompts/problems x, π_θ is our model, y is a sampled response/solution to x, and $r(y \mid x)$ denotes whether y is a correct answer to x (we will also call r our "reward function"). Note that this objective is quite different from the pretraining objective we've been studying so far in the class: unlike the cross entropy loss $E_{x\sim\mathcal{D}}[-\log \pi_\theta(x)]$, where we draw samples from a dataset $\mathcal{D}$, our accuracy objective involves an expectation over samples from the model itself. Therefore, we will need new tools to optimize this objective: in particular, we will need to use reinforcement learning (RL). At a high level, RL will involve sampling responses from the model, scoring them with a grading function, and reinforcing the ones that are correct. RL is often slow and challenging, so much of this assignment will involve exploring tools to make it faster and more stable.

One other aspect of RL is that because it involves sampling from the model, its dynamics are shaped by the prompt one uses during sampling. In particular, one exciting finding in language model research is that when using a chain-of-thought reasoning prompt, performing RL on a strong base model can lead to substantial improvements in its reasoning capabilities [OpenAI et al., 2024; DeepSeek-AI et al., 2025]. Therefore, in this assignment, in addition to exploring core algorithmic ideas in RL, we will also explore its interactions with language modeling techniques like chain-of-thought prompting.

4.1 Deriving on-policy GRPO

We will first explore the math behind RL, building up to the Group Relative Policy Optimization (GRPO) algorithm, a commonly used RL algorithm when training language models [Z. Shao et al., 2024].

Our presentation is based closely on a couple of great resources which walk through these concepts in more depth: OpenAI's Spinning Up in Deep RL [J. Achiam, 2018] and Nathan Lambert's Reinforcement Learning from Human Feedback (RLHF) Book [N. Lambert, 2024].

4.1.1 Language models as policies

Traditionally in reinforcement learning, we think about a *policy* which takes in a *state* s_t and outputs an *action* a_t. This action then results in some *reward* $r_t = r(s_t, a_t)$, and a subsequent state drawn from some next state distribution $s_{t+1} \sim \text{next_state}(s_t, a_t)$. Reinforcement learning then involves optimizing the policy to achieve high reward.

A causal language model (LM) π_θ with parameters θ defines a probability distribution over the next token y_t given the current text prefix $y_{<t} = (y_1, ..., y_{t-1})$. So in the context of RL, we can think of the next token y_t as our *action* a_t and the current text prefix $y_{<t}$ as the *state* s_t. Hence, the LM is a *categorical stochastic policy*

$$a_t \sim \pi_\theta(\cdot \mid s_t), \pi_\theta(a_t \mid s_t) = \left[\text{softmax}\left(f_\theta(s_t)\right)\right]_{a_t}. \tag{2}$$

We will need two primitive operations when optimizing a policy with policy gradients:

1. *Sampling from the policy*: drawing an action a_t from the categorical distribution above;
2. *Scoring the log-likelihood of an action*: evaluating $\log \pi_\theta(a_t \mid s_t)$.

Generally, when doing RL with LLMs, s_t is the partial completion/solution produced so far, and each a_t is the next token of the solution; the episode ends when an end-of-text token is emitted, like `<| end_of_text|>`, or `</answer>` in the case of our `r1_zero` prompt.

4.1.2 Trajectories

In reinforcement learning, we start at an initial state drawn from some starting distribution $s_0 \sim \rho$. We then sample an action a_t from the policy π_θ, transition to the next state according to some next state distribution $s_{t+1} \sim \text{next_state}(s_t, a_t)$, and repeat. This series of states and actions then forms a (finite-horizon) *trajectory*:

$$\tau = (s_0, a_0, s_1, a_1, ..., s_T, a_T), \tag{3}$$

where T is the length of the trajectory, i.e., a_T is an end-of-text token or we have reached a maximum generation budget in tokens.

In our case, $s_0 \sim \rho$ is our prompt x containing the math problem we want to solve (possibly with CoT prompting or few-shot applied). Starting with this prefix, we then sample tokens one by one, and the next state is the old prefix concatenated with the emitted token, or $s_{t+1} = (s_t, a_t)$. Trajectories are also called *episodes* or *rollouts*; we will use these terms interchangeably.

4.1.3 Rewards and return

In RL, a scalar reward $r_t = r(s_t, a_t)$ judges the immediate quality of the action taken at state s_t. For RL on verified domains, like our task of mathematical word problems, we observe zero reward for intermediate reasoning steps, until we output the answer. We then receive a *verified reward* on the terminal action

$$r_T = r(s_T, a_T) := \begin{cases} 1 \text{ if trajectory } \tau \text{ matches the ground-truth according to our reward function} \\ 0 \text{ otherwise} \end{cases}. \tag{4}$$

In our case, r_T is 1 if our final answer is correct, and 0 otherwise. The *return* $R(\tau)$ aggregates rewards along the trajectory, and in our case is just r_T.

The objective of the agent is to maximize the expected return

$$J_\theta = E_{\tau \sim \pi_\theta}[r(\tau)], \tag{5}$$

where $\tau \sim \pi_\theta$ is the distribution over trajectories where we first sample $s_0 \sim \rho$, and we then sample actions from the policy and the next states from the next state distribution. In our case, this means first sampling our math problem $x \sim \rho$ from our dataset, and then sampling tokens until we hit the end of our response, a process we can denote by $y \sim \pi_\theta(y \mid x)$. This objective leads to the optimization problem

$$\theta^* = \arg\max_\theta J_\theta. \tag{6}$$

4.1.4 Policy gradients

We now have the notation and definitions we need to learn an RL algorithm called *policy gradients*. In the remainder of the assignment, we will stick to the language model notation (x, y) denoting prompts and responses, instead of the RL notation of states s_t and actions a_t.

Recall that we want to optimize the model's expected reward (i.e. accuracy), given by

$$J_\theta = E_{x \sim \rho} E_{y \sim \pi_\theta(y \mid x)}[r(y \mid x)], \tag{7}$$

where $r(y \mid x)$ denotes whether response y is correct for prompt x. Our approach will be to do gradient ascent on the objective, or

$$\theta_{k+1} = \theta_k + \alpha \nabla_\theta J_{\theta_k}, \tag{8}$$

but to do so, we need to be able to estimate the gradient $\nabla_\theta J_{\theta_k}$ from samples. Rewriting this gradient as an expectation over samples will produce an expression known as the "REINFORCE policy gradient", named after the REINFORCE paper [R. J. Williams, 1992]. In particular, we'll see that we can rewrite the gradient as follows:

$$\nabla_\theta E_{x \sim \rho} E_{y \sim \pi_\theta(y \mid x)}[r(y \mid x)] = E_{x \sim \rho} E_{y \sim \pi_\theta(y \mid x)}[r(y \mid x) \nabla_\theta \log \pi_\theta(y \mid x)]. \tag{9}$$

Before walking through the derivation, note that this expression implies an intuitive algorithm (REINFORCE): we can sample problems x from our dataset, sample responses y from our model, compute the average of $r(y \mid x) \nabla_\theta \log \pi_\theta(y \mid x)$ over these samples, and then take a gradient step. The expression $r(y \mid x) \nabla_\theta \log \pi_\theta(y \mid x)$ says that we should upweight the log probability of responses y with high reward $r(y \mid x)$. If we repeat this procedure and take multiple gradient steps, we have a basic reinforcement learning training loop.

To derive the REINFORCE policy gradient, we can use the log-derivative trick (recall that the derivative of $\log f(x)$ is $f' \frac{x}{f(x)}$):

$$\nabla_\theta \pi_\theta(y \mid x) = \pi_\theta(y \mid x) \nabla_\theta \log \pi_\theta(y \mid x). \tag{10}$$

Applying this identity directly produces the desired result:

$$\begin{aligned}
\nabla_\theta E_{y \sim \pi_\theta(y \mid x)}[r(y \mid x)] &= \sum_y \nabla_\theta \pi_\theta(y \mid x) r(y \mid x) && (11)\\
&= \sum_y \pi_\theta(y \mid x) \nabla_\theta \log \pi_\theta(y \mid x) r(y \mid x) && (12)\\
&= E_{y \sim \pi_\theta(y \mid x)}[r(y \mid x) \nabla_\theta \log \pi_\theta(y \mid x)]. && (13)
\end{aligned}$$

Note that in these equations, we omitted the expectation over prompts $E_{x \sim \rho}$ because ∇_θ and $E_{x \sim \rho}$ can be interchanged.

So given a batch of prompts $x^{(1)}, ..., x^{(B)} \overset{\text{iid}}{\sim} \rho$ and G responses per prompt $y^{(i,j)} \overset{\text{iid}}{\sim} \pi_\theta(y \mid x^{(i)})$, we have the following sample estimate of the policy gradient:

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^{B} \sum_{j=1}^{G} r(y^{(i,j)} \mid x^{(i)}) \nabla_\theta \log \pi_\theta(y^{(i,j)} \mid x^{(i)}), \tag{14}$$

which upweights responses with high reward. To derive GRPO, we will start with this basic policy gradient estimator and then apply modifications one by one.

4.1.5 Baselines

While the basic REINFORCE policy gradient estimator has the correct expectation $\nabla_\theta J_\theta$, it has high variance, making RL unstable. To stabilize training and reduce variance, one tool people use is *baselines*.

While baselines can get quite complicated, the simplest baseline is to subtract a constant b from the reward:

$$E_{x \sim \rho} E_{y \sim \pi_\theta(y \mid x)}[(r(y \mid x) - b) \nabla_\theta \log \pi_\theta(y \mid x)]. \tag{15}$$

For example, if we subtract $b = 0.5$ and our reward r is binary (1 for correct, 0 for incorrect), using this estimator means we sample responses from the model, upweight correct responses (with weight 0.5), and downweight incorrect responses (with weight -0.5). This is in contrast to our original gradient estimator, which upweights correct responses with weight 1 and does nothing for incorrect responses (with weight 0).

The key piece of math for this section tells us that subtracting a baseline b preserves the expectation of the estimator, as long as b does not depend on the action/response y (e.g. it can be a constant, or a function of x, or a function of the policy). As long as b does not depend on y, subtracting it in our estimator preserves its expectation:

$$E_{x \sim \rho} E_{y \sim \pi_\theta(y \mid x)}[(r(y \mid x) - b) \nabla_\theta \log \pi_\theta(y \mid x)] = E_{x \sim \rho} E_{y \sim \pi_\theta(y \mid x)}[r(y \mid x) \nabla_\theta \log \pi_\theta(y \mid x)]. \tag{16}$$

This fact follows directly from the following identity:

$$E_{y \sim \pi_\theta(y \mid x)}[\nabla_\theta \log \pi_\theta(y \mid x)] = \nabla_\theta E_{y \sim \pi_\theta(y \mid x)}[1] = 0, \tag{17}$$

where the first step follows from applying our previous log-derivative trick in reverse.

While baselines preserve the expectation of the estimator, they can increase or decrease variance. In notation, "variance" means the following: writing our policy gradient estimator as a sample mean $\frac{1}{n} \sum_{i=1}^{n} Z_i$, where each Z_i is our expression $(r(y \mid x) - b) \nabla_\theta \log \pi_\theta(y \mid x)$ for a sample (x, y), the variance of our estimator is $\frac{E[Z_i^2] - E[Z_i]^2}{n}$. We'll explore when baselines increase or decrease variance in the following problem.

Problem (`baseline_calcs`): Compute the variance of the policy gradient estimator (5 points)

Let π_θ define a policy over the binary action space $\mathcal{A} = \{0, 1\}$ with $\pi_{\theta(A=1)} = p = \sigma(\theta)$, where σ represents the sigmoid function $\sigma(\theta) = \frac{1}{1+e^{-\theta}}$. Our binary reward function will assign reward 1 to action $A = 1$ and reward 0 otherwise, or $r(A) = \mathbb{1}\{A = 1\}$.

(a) Let the policy gradient estimator be given by

$$\frac{1}{n} \sum_{i=1}^{n} r(A_i) \nabla_\theta \log \pi_\theta(A_i) \tag{18}$$

given n samples $A_i \overset{\text{iid}}{\sim} \pi_\theta$. What is the variance of this estimator?

Deliverable: An expression in terms of n and p, accompanied by a derivation.

(b) Let the baseline-adjusted policy gradient estimator be given by

$$\frac{1}{n}\sum_{i=1}^{n}(r(A_i)-b)\nabla_\theta \log \pi_\theta(A_i) \tag{19}$$

given n samples $A_i \overset{\text{iid}}{\sim} \pi_\theta$. What is the variance of this estimator?

Deliverable: An expression in terms of n, b, and p, accompanied by a derivation and some discussion.

(c) What is the resulting variance if we substitute the "population mean" baseline $b = p$? Compare this variance to that of the unadjusted policy gradient estimator: is it always lower, always higher, or sometimes higher or lower depending on p?

Baselines in GRPO

The first component of GRPO will be to apply the "group mean" baseline:

$$\hat{g} \leftarrow \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\left(r\left(y^{(i,j)} \mid x^{(i)}\right)-\mu_i\right)\nabla_\theta \log \pi_\theta\left(y^{(i,j)} \mid x^{(i)}\right) \tag{20}$$

for $\mu_i = \frac{1}{G}\sum_{j=1}^{G} r\left(y^{(i,j)} \mid x^{(i)}\right)$, i.e. the average reward of the model on prompt $x^{(i)}$. Note that even though μ_i depends on the responses y, it still preserves the expectation up to a rescaling $\frac{G-1}{G}$:

$$E_{y^{(1)},\ldots,y^{(G)}\sim\pi_\theta(y\mid x)}\left[\frac{1}{G}\sum_{j=1}^{G}\left(r\left(y^{(j)} \mid x\right)-\mu_i\right)\nabla_\theta \log \pi_\theta\left(y^{(j)} \mid x\right)\right] \tag{21}$$

$$= \frac{1}{G}\sum_{j=1}^{G} E_{y^{(2)},\ldots,y^{(G)}\sim\pi_\theta(y\mid x)}\left[E_{y^{(1)}\sim\pi_\theta(y\mid x)}\left[\left(r\left(y^{(1)} \mid x\right)-\mu_i\right)\nabla_\theta \log \pi_\theta\left(y^{(1)} \mid x\right)\right]\right] \tag{22}$$

$$= \frac{1}{G}\sum_{j=1}^{G} E_{y^{(1)}\sim\pi_\theta(y\mid x)}\left[\left(r\left(y^{(1)} \mid x\right)-\frac{1}{G}r\left(y^{(1)} \mid x\right)\right)\nabla_\theta \log \pi_\theta\left(y^{(1)} \mid x\right)\right] \tag{23}$$

$$= \frac{G-1}{G}E_{y\sim\pi_\theta(y\mid x)}[r(y \mid x)\nabla_\theta \log \pi_\theta(y \mid x)]. \tag{24}$$

These group-mean-adjusted rewards are often called *advantages*: instead of the absolute reward, this term represents the "advantage" of this rollout relative to the average reward.

4.1.6 Advantage normalization

The next modification in GRPO, after subtracting the group mean, will be to divide by the group standard deviation:

$$\hat{g} \leftarrow \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\frac{r\left(y^{(i,j)} \mid x^{(i)}\right)-\mu_i}{\text{std}_i}\nabla_\theta \log \pi_\theta\left(y^{(i,j)} \mid x^{(i)}\right), \tag{25}$$

where $\text{std}_i = \sqrt{\frac{1}{G}\sum_{j=1}^{G}\left(r\left(y^{(i,j)} \mid x^{(i)}\right)-\mu_i\right)^2}$ (note that the default `torch.std` function uses a slightly different sample std expression to do bias correction; you should use the default `torch.std` in your implementation). Dividing by the standard deviation no longer preserves the expectation of the estimator, so we're no longer doing gradient ascent on our original objective J_θ (expected reward). One way to think about this modification is that it is a stability trick: if we assume each gradient vector is iid Gaussian,

dividing by the group std ensures that each group's gradient update is roughly equal in norm. We'll dive deeper into advantage normalization in the next section.

4.1.7 Sequence normalization

There's one other detail in GRPO that seems to be inherited from earlier LLM-PPO implementations, called *sequence normalization*. Recall that our policy gradient estimator (so far) is the following (where we've expanded $\log \pi_\theta\big(y^{(i,j)} \mid x^{(i)}\big)$ into a sum over time steps):

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^{B} \sum_{j=1}^{G} \frac{r\big(y^{(i,j)} \mid x^{(i)}\big) - \mu_i}{\text{std}_i} \nabla_\theta \sum_{t=1}^{\text{len}\left(y^{(i,j)}\right)} \log \pi_\theta\Big(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)}\Big). \tag{26}$$

GRPO adds an additional sequence length normalization term:

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^{B} \sum_{j=1}^{G} \frac{r\big(y^{(i,j)} \mid x^{(i)}\big) - \mu_i}{\text{std}_i} \nabla_\theta \left(\frac{1}{\text{len}\big(y^{(i,j)}\big)} \sum_{t=1}^{\text{len}\left(y^{(i,j)}\right)} \log \pi_\theta\Big(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)}\Big) \right). \tag{27}$$

Note that this modification changes the expectation further: instead of weighting all tokens equally across sequences, tokens in longer sequences are downweighted relative to tokens in shorter sequences. In the next section, we'll discuss this choice and do ablations to see whether it is a good modification. But for now, we'll just implement the standard GRPO algorithm, as presented in the original paper, so we will include sequence normalization.

4.1.8 Putting everything together

There is one component of GRPO we haven't talked about yet, namely clipped importance reweighting. We will discuss this component later in the assignment, when we discuss off-policy RL. Briefly, off-policy RL refers to the approach of speeding up RL by taking multiple gradient steps per inference batch, in contrast with on-policy RL where we take one gradient step per inference batch. The term "off-policy" refers to the fact that for the second gradient step onward, samples in the inference batch are now stale, in that they come from an older version of the model. Importance reweighting is then a modification to the gradient estimator that allows it to use stale samples. For now, we will only do on-policy RL, so we won't need importance reweighting yet.

We now have all the components we need to implement on-policy GRPO. The algorithm is provided in Algorithm 1.

Algorithm 1: On-policy Group Relative Policy Optimization (GRPO)

Input initial policy model π_{θ_0}; reward function r; task distribution (or dataset) ρ; learning rate α

Output π_θ

policy model $\pi_\theta \leftarrow \pi_{\theta_0}$

For step = 1, …, `n_grpo_steps`

Sample a batch of questions $x^{(1)}, \ldots, x^{(B)} \overset{\text{iid}}{\sim} \rho$

Sample G outputs per question: $y^{(i,1)}, \ldots, y^{(i,G)} \overset{\text{iid}}{\sim} \pi_\theta\big(y \mid x^{(i)}\big)$

Compute the on-policy GRPO policy gradient estimator

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^{B} \sum_{j=1}^{G} \frac{1}{\operatorname{len}\big(y^{(i,j)}\big)} \sum_{t=1}^{\operatorname{len}\left(y^{(i,j)}\right)} \frac{r\big(y^{(i,j)} \mid x^{(i)}\big) - \mu_i}{\operatorname{std}_i} \nabla_\theta \log \pi_\theta\Big(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)}\Big), \tag{28}$$

where

$$\mu_i = \frac{1}{G} \sum_{j=1}^{G} r\big(y^{(i,j)} \mid x^{(i)}\big) \tag{29}$$

denotes the group mean and

$$\operatorname{std}_i = \sqrt{\frac{1}{G} \sum_{j=1}^{G} \big(r\big(y^{(i,j)} \mid x^{(i)}\big) - \mu_i\big)^2} \tag{30}$$

denotes the group standard deviation.

Update θ with your favorite optimizer (we provide the stochastic gradient ascent update below):

$$\theta \leftarrow \theta + \alpha \hat{g}. \tag{31}$$

4.2 Implementing on-policy GRPO

4.2.1 Using Hugging Face models

While previous assignments used our own language model implementation in `cs336_basics`, in this assignment we will use the Hugging Face transformers library directly to load the pre-trained base model. You're also welcome to implement your own transformer and pre-trained weight loader if you would like; just make sure the architecture matches that of OLMo-2-0425-1B.

To load a Hugging Face model and tokenizer (in `bfloat16` and with FlashAttention-2 to save memory), you can use the following starter code, available in `cs336_alignment/checkpoint.py`:

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

def get_model_and_tokenizer(model_id_or_dir: str, device: str):
    model = AutoModelForCausalLM.from_pretrained(
        model_id_or_dir,
        device_map=device,
        torch_dtype=torch.bfloat16,
        attn_implementation="eager" if device=='cpu' else "flash_attention_2",
    )
```

```
    tokenizer = AutoTokenizer.from_pretrained(model_id_or_dir)
    return model, tokenizer
```

`model_id_or_directory` can be a model name like `allenai/OLMo-2-0425-1B` or the path to a directory. The directory will typically be the result of calling `save_pretrained`, which you can call to save your trained model:

```
# Save the model weights
model.save_pretrained(save_directory=output_dir)
tokenizer.save_pretrained(save_directory=output_dir)
```

First, let's implement a helper function which uses the pre-trained tokenizer to tokenize input prompts and responses. Tokenize the prompt and response separately without adding special tokens, then concatenate them directly as `prompt_ids + response_ids` with no EOS, BOS, or separator token between them. We will also need to construct a `response_mask`: a boolean mask, aligned with the shifted `labels`, that is `True` exactly where the label token comes from the response and `False` for prompt and padding positions. We will use this mask in the training loop to ensure that we only compute the loss on the response tokens.

Problem (`tokenize_prompt_and_output`): Prompt and output tokenization (1 point)

Deliverable: Implement a method `tokenize_prompt_and_output` that tokenizes the prompt and output separately, concatenates them together without inserting special tokens, and constructs a `response_mask`. The following interface is recommended:

```
def tokenize_prompt_and_output(
    prompt_strs: list[str],
    output_strs: list[str],
    tokenizer: PreTrainedTokenizer,
) -> dict[str, torch.Tensor]:
```

Tokenize the prompt and output strings, and construct a mask aligned with `labels` that is 1 for response tokens and 0 for other tokens (prompt or padding).

Args:

- **`prompt_strs: list[str]`** List of prompt strings.
- **`output_strs: list[str]`** List of output strings.
- **`tokenizer: PreTrainedTokenizer`** Tokenizer to use for tokenization.

Returns:

- **`dict[str, torch.Tensor].`** Let `prompt_and_output_lens` be a list containing the lengths of the concatenated tokenized prompt and output strings. Then the returned dictionary should have the following keys:
  - **`input_ids`** torch.Tensor of shape `(batch_size, max(prompt_and_output_lens) - 1)`: the tokenized prompt and output strings, with the final token sliced off.
  - **`labels`** torch.Tensor of shape `(batch_size, max(prompt_and_output_lens) - 1)`: shifted input ids, i.e., the input ids without the first token.

- **response_mask** torch.Tensor of shape `(batch_size, max(prompt_and_output_lens) - 1)`: a mask aligned with `labels`, with value 1 where the corresponding label token is part of the response and 0 otherwise.

To test your code, implement [**adapters.run_tokenize_prompt_and_output**] . Then, run the test with `uv run pytest -k test_tokenize_prompt_and_output` and make sure your implementation passes it.

Once we have a tokenized input, we can pass it through the model as follows:

```
input_ids = train_batch["input_ids"].to(device)
labels = train_batch["labels"].to(device)
logits = model(input_ids).logits
```

With this syntax in hand, please implement the following method, which computes the per-token log-probabilities for a response under the model, a primitive we'll need for computing our policy gradient. In RL, it's often also useful to log the per-token entropies, so this function will include a `return_token_entropy` option you will need to implement as well.

Problem (`get_response_log_probs`): Response log-probs (and entropy) (1 point)

Deliverable: Implement a method `get_response_log_probs` that gets per-token conditional log-probabilities (given the previous tokens) from a causal language model, and optionally the entropy of the model's next-token distribution.

The following interface is recommended:

```
def get_response_log_probs(
    model: PreTrainedModel,
    input_ids: torch.Tensor,
    labels: torch.Tensor,
    return_token_entropy: bool = False,
) -> dict[str, torch.Tensor]:
```

Args:

- **model: PreTrainedModel** HuggingFace model used for scoring (placed on the correct device and in inference mode if gradients should not be computed).
- **input_ids: torch.Tensor** shape `(batch_size, sequence_length)`, concatenated prompt + response tokens as produced by your tokenization method.
- **labels: torch.Tensor** shape `(batch_size, sequence_length)`, labels as produced by your tokenization method.
- **return_token_entropy: bool** If `True`, also return per-token entropy.

Returns:

- **dict[str, torch.Tensor].**
  - **"log_probs"** shape `(batch_size, sequence_length)`, conditional log-probabilities $\log p_\theta(x_t \mid x_{<t})$.
  - **"token_entropy"** optional, shape `(batch_size, sequence_length)`, per-token entropy for each position (present only if `return_token_entropy=True`).

To test your code, implement [`adapters.run_get_response_log_probs`] . Then run `uv run pytest -k test_get_response_log_probs` and ensure the test passes.

4.2.2 Using vLLM in a reinforcement learning loop

In our RL loop, we'll also need to generate rollouts. The specific configuration we'll use is to put the Hugging Face model and optimizer on one GPU for training, and vLLM (which includes the model and kv cache) on the other GPU. So in addition to the vLLM initialization and generation functions described in the previous section, we'll also need to sync weights between the two devices before each inference step. We describe the weight sync code below, which is available in `cs336_alignment/vllm_utils.py`.

```
@dataclass
class VLLMServer:
    gpu: int = 1 # Run training on gpu 0 and inference on gpu 1

    # Create the NCCL weight-transfer group between the training GPU and vLLM.
    def init_weight_sync(self, policy_device: str): ...

    # Copy the current Hugging Face policy weights into the vLLM server and
    # reset vLLM caches that depended on the old weights.
    def sync_policy_weights(self, policy: torch.nn.Module) -> None: ...

    # Generate rollouts from the current vLLM weights.
    def generate_completions(
        self,
        prompts: list[str],
        sampling_params: dict,
        batch_size: int | None = None,
    ) -> list[VLLMCompletion]: ...
```

Like our prompting experiments, we'll sample with temperature 1.0, top-p 1.0, max generation length 512. The prompt asks the model to end its answer with the string `</answer>`, so we can direct vLLM to stop when the model outputs this string:

```
# Based on Dr. GRPO: stop when the model completes its answer
# https://github.com/sail-sg/understand-r1-zero/blob/
#   c18804602b85da9e88b4aeeb6c43e2f08c594fbc/train_zero_math.py#L167
sampling_params['stop'] = ["</answer>"]
sampling_params['include_stop_str_in_output'] = True
```

4.2.3 GRPO components

Next, let's implement components that compute parts of the GRPO loss. Note that later on in the assignment, we will implement variants, like different advantage normalizers or importance reweighting approaches. The components below are designed so that we can swap between variants with minimal code overlap, so we've provided interfaces with arguments that specify which variant. For this part of the assignment, you only need to implement the standard GRPO variant.

Our first step will be to implement a helper function that computes the rewards of responses.

Problem (`compute_rollout_rewards`): Computing the rewards of rollouts (1 point)

Deliverable: Implement a method `compute_rollout_rewards` that calculates raw rewards for each rollout response.

The following interface is recommended:

```
def compute_rollout_rewards(
    reward_fn: Callable[[str, str], dict[str, float]],
    rollout_responses: list[str],
    repeated_ground_truths: list[str],
) -> tuple[torch.Tensor, dict[str, float]]:
```

Compute rewards for a list of rollout responses, along with metadata for the reward components.

Args:

- **`reward_fn: Callable[[str, str], dict[str, float]]`** Scores the rollout responses against the ground truths, producing a dict with keys `"reward"`, `"format_reward"`, and `"answer_reward"`.
- **`rollout_responses: list[str]`** Rollouts from the policy. The length of this list is `rollout_batch_size = n_prompts_per_rollout_batch * group_size`.
- **`repeated_ground_truths: list[str]`** The ground truths for the examples. The length of this list is `rollout_batch_size`, because the ground truth for each example is repeated `group_size` times.

Returns:

- **`tuple[torch.Tensor, dict[str, float]]`.**
  - **`raw_rewards`** shape `(rollout_batch_size,)`. Unnormalized rewards for each rollout response.
  - **`metadata`** Reward statistics to log. At minimum, include the mean total and format rewards over the rollout batch.

To test your code, implement [**`adapters.run_compute_rollout_rewards`**] . Then, run the test with `uv run pytest -k compute_rollout_rewards` and make sure your implementation passes it.

Next, let's implement the core math in GRPO, which normalizes these rewards to turn them into advantages. Note that the rewards (output by our function above) are flattened, so we need to take in the group size to reshape it back into groups to do normalization. To pass the test, you'll need to support `baseline = "mean"` with `advantage_normalizer = "std"`. To avoid division by zero, you should also add `advantage_eps` to the per-group std before dividing.

Problem (`compute_group_normalized_rewards_grpo`): Group normalization (1 point)

Deliverable: Implement a method `compute_group_normalized_rewards` that normalizes raw rewards within their groups and returns the normalized rewards along with any metadata you think is useful.

For now, you only need to support `baseline = "mean"` and `advantage_normalizer = "std"`. Feel free to raise a `NotImplementedError` for unsupported inputs. In later parts of the assignment we will implement the other options and run ablations. Remember to add `advantage_eps` to the normalizer to avoid division by zero.

The following interface is recommended:

```python
def compute_group_normalized_rewards(
    raw_rewards: torch.Tensor,
    group_size: int,
    baseline: Literal["mean", "none"] = "mean",
    advantage_eps: float = 1e-6,
    advantage_normalizer: Literal["std", "none", "mean"] = "std",
):
```

Compute advantages by applying the requested baseline and normalization within each group.

Args:

- **`raw_rewards: torch.Tensor`** shape `(rollout_batch_size,)`. Unnormalized rewards for each rollout response, where `rollout_batch_size = n_prompts_per_rollout_batch * group_size`.
- **`group_size: int`** Number of responses per question (group).
- **`baseline: Literal["mean", "none"]`** For this problem, support `mean`, which subtracts the per-group mean reward. Later, `none` will mean no baseline subtraction.
- **`advantage_eps: float`** Small constant to avoid division by zero in normalization.
- **`advantage_normalizer: Literal["std", "none", "mean"]`** For this problem, support `std`, which divides by the per-group standard deviation. Later, `none` will mean no normalization and `mean` will mean divide by the per-group mean reward.

Returns:

- **`tuple[torch.Tensor, dict[str, float]]`.**
  - **`advantages`** shape `(rollout_batch_size,)`. Group-normalized rewards for each rollout response.
  - **`metadata`** your choice of other statistics to log (e.g. mean, std, max/min of rewards).

To test your code, implement [**`adapters.run_compute_group_normalized_rewards`**] . Then, run the test with `uv run pytest -k compute_group_normalized_rewards_grpo` and make sure your implementation passes it.

The next step is to implement a function which computes the policy gradient loss per token. Note: the equation in Algorithm 1 writes out the gradient estimator, but this problem asks you to write a per-token loss such that taking the gradient produces each term of the GRPO gradient estimator (which we will then aggregate over tokens). This expression is not a loss in the traditional sense, where we expect it to go down during training. It's just an expression such that when we take the gradient, we get the policy gradient.

The specific loss you should implement is

$$J_\theta^{\text{GRPO-on-policy}} = \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\frac{1}{\text{len}(y^{(i,j)})}\sum_{t=1}^{\text{len}(y^{(i,j)})}\frac{r\left(y^{(i,j)} \mid x^{(i)}\right)-\mu_i}{\text{std}_i}\log \pi_\theta\left(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)}\right), \tag{32}$$

which you can check produces the gradient term in Algorithm 1 when differentiated. Since PyTorch optimizers perform gradient descent, your implementation should return the negative of this objective as the loss. `compute_policy_gradient_loss` will compute the loss term per token and per sequence, while `aggregate_loss_across_microbatch_sequence` will average the per-token losses across tokens and sequences.

Problem (`compute_policy_gradient_loss_on_policy`): On-policy policy gradient (1 point)

Deliverable: Implement a method `compute_policy_gradient_loss` that computes the per-token policy-gradient loss given raw rewards or pre-computed advantages.

For now, we will assume all rollouts are on-policy, so you only need to support `importance_reweighting_method = "none"`, and you can ignore the old-log-prob and clipping arguments. Feel free to raise a `NotImplementedError` for unsupported inputs. We will implement clipping later in the assignment.

The following interface is recommended:

```
def compute_policy_gradient_loss(
    raw_rewards_or_advantages: torch.Tensor,
    policy_log_probs: torch.Tensor,
    importance_reweighting_method: Literal["none", "noclip", "grpo", "gspo"] = "none",
    old_log_probs: torch.Tensor | None = None,
    cliprange: float | None = None,
    response_mask: torch.Tensor | None = None,
) -> tuple[torch.Tensor, dict[str, torch.Tensor]]:
```

Compute the policy-gradient loss at every token, where `raw_rewards_or_advantages` is either the raw reward or an already-normalized advantage.

Args:

- **`raw_rewards_or_advantages: torch.Tensor`** Shape `(batch_size,)` or `(batch_size, 1)`, scalar reward/advantage for each rollout response.
- **`policy_log_probs: torch.Tensor`** Shape `(batch_size, sequence_length)`, logprobs for each token.
- **`importance_reweighting_method: Literal["none", "noclip", "grpo", "gspo"]`** `"none"`: no importance reweighting; `"noclip"`: apply importance reweighting without clipping; `"grpo"`: do PPO/GRPO-style token-level reweighting and clipping; `"gspo"`: do GSPO-style sequence-level reweighting and clipping.
- **`old_log_probs: torch.Tensor | None`** Required unless `importance_reweighting_method = "none"`; shape `(batch_size, sequence_length)`.
- **`cliprange: float | None = None`** Clip parameter ε, required when `importance_reweighting_method` is `"grpo"` or `"gspo"`.
- **`response_mask: torch.Tensor | None = None`** Optional shape `(batch_size, sequence_length)` mask over response tokens. Required for GSPO implementations that average the sequence-level log-ratio over response tokens only.

Returns:

- **`tuple[torch.Tensor, dict[str, torch.Tensor]]`.**
  - **`per_token_policy_gradient_loss`** Shape `(batch_size, sequence_length)`, the per-token policy-gradient loss (to be aggregated across the batch and sequence dimensions in the training loop).
  - **`metadata`** Statistics from the underlying loss call, such as clip-fraction components.

To test your code, implement [adapters.run_compute_policy_gradient_loss] . Then run `uv run pytest -k test_compute_policy_gradient_loss_on_policy` and ensure the test passes.

Finally, let's implement the aggregation function over tokens and sequences, which in standard GRPO involves averaging over tokens in each sequence, and then averaging over sequences. Later on, we'll implement the alternative approach of just dividing by a constant, which we saw in the derivation is more faithful to the original policy gradient we were trying to estimate.

Problem (`aggregate_loss_across_microbatch_sequence`): Aggregate loss across tokens and sequences (0.5 points)

Deliverable: Implement a method `aggregate_loss_across_microbatch` that takes in the per-token policy-gradient loss and the response mask, and outputs the average loss. For now, we will use standard GRPO aggregation, which involves first averaging within each sequence, and then averaging over sequences, so you can assume `loss_normalization = "sequence"`. Feel free to raise a `NotImplementedError` for unsupported inputs.

The following interface is recommended:

```
def aggregate_loss_across_microbatch(
    per_token_policy_gradient_loss: torch.Tensor,
    mask: torch.Tensor,
    loss_normalization: Literal["sequence", "constant"] = "sequence",
    normalization_constant: int | None = None,
) -> torch.Tensor:
```

Aggregate the per-token policy-gradient loss according to the response mask and loss-normalization strategy.

Args:

- **`per_token_policy_gradient_loss: torch.Tensor`** Shape `(batch_size, sequence_length)`, the per-token policy-gradient loss (to be aggregated across the batch and sequence dimensions in the training loop).
- **`mask`** torch.Tensor of shape `(batch_size, sequence_length)` denoting which positions should be included in the loss.
- **`loss_normalization: Literal["sequence", "constant"] = "sequence"`** `"sequence"`: average loss over each sequence, then average over sequences; `"constant"`: normalize total loss by a constant.
- **`normalization_constant: int | None = None`** The constant to divide total loss by; required if `loss_normalization = "constant"`.

Returns:

- **`loss: torch.Tensor`** A scalar containing the average loss. Make sure you can later call backward on this loss.

To test your code, implement [adapters.run_aggregate_loss_across_microbatch] . Then run `uv run pytest -k test_aggregate_loss_across_microbatch_sequence` and ensure the test passes.

4.2.4 GRPO training step

We're now ready to piece these components together into a full training step.

Gradient accumulation

We need a large batch size to get good utilization during inference. As a result, in on-policy RL, where we set our train batch size equal to our inference batch size, our GPU will not have enough memory to compute the gradient on the entire batch at once. Therefore, we'll need to split the batch into a series of *microbatches* and accumulate the gradient across these microbatches. The main tricky part is to handle normalization properly to ensure that the microbatch-accumulated gradient is equivalent to computing the gradient on the whole batch.

Gradient accumulation is straightforward to implement in PyTorch. Recall that each weight tensor has an attribute `.grad` that stores its gradient. Before we call `loss.backward()`, the `.grad` attribute is None. After we call `loss.backward()`, the `.grad` attribute contains the gradient. Normally, we'd take an optimizer step, and then zero the gradients with `optimizer.zero_grad()`, which resets the `.grad` field of the weight tensors:

```
# Forward pass.
logits = model(inputs)
loss = loss_fn(logits, labels)

# Backward pass.
loss.backward()

# Update weights.
optimizer.step()
# Zero gradients in preparation for next iteration.
optimizer.zero_grad()
```

To implement gradient accumulation, we'll split the batch into k microbatches, and just call `optimizer.step()` and `optimizer.zero_grad()` once. Assuming sequence normalization, where we first average loss over each sequence and then across sequences, after computing the average microbatch loss we'll also need to reweight by the number of sequences.

```
gradient_accumulation_steps = 4
microbatch_size = len(inputs) // gradient_accumulation_steps
for i in range(0, len(inputs), microbatch_size):
    inputs_microbatch = inputs[i:i+microbatch_size]
    labels_microbatch = labels[i:i+microbatch_size]

    # Forward pass.
    logits = model(inputs_microbatch)
    loss = loss_fn(logits, labels_microbatch) * (len(inputs_microbatch) / len(inputs))

    # Backward pass.
    loss.backward()
# Update weights once across entire batch.
optimizer.step()
# Zero gradients once across entire batch.
optimizer.zero_grad()
```

Note that later on when we do constant normalization instead of sequence normalization, the microbatch code will change slightly to account for the different normalization.

Implementing the train step

In the following function, we'll finally implement a GRPO training step, given some batch of rollouts. The function will take in the model, tokenizer, optimizer, reward function, prompts and rollouts, and various hyperparameters, and it will accumulate gradients and take an optimizer step. You will need to implement gradient accumulation, as described above, to not run out of memory. Before the optimizer step, the function should also clip the gradient norm to `max_grad_norm`. The function should then return the training loss on the batch, along with metadata, both for logging. Please log at least the following metrics in your train step:

- The loss.
- Gradient norm.
- Token entropy.
- Train rewards (total, format).

You're welcome to log other metrics as well: for example, it might be useful to log `pass@k` for `k` between 1 and `group_size` (i.e. the fraction of prompts for which there is at least one correct answer within the first `k` rollouts).

Problem (`grpo_train_step_standard_on_policy`): GRPO train step (5 points)

Deliverable: Implement a single batch update for policy gradients, given the model, tokenizer, and rollouts. For this part of the assignment, you only need to implement this function for standard GRPO in the on-policy setting, or `baseline = "mean"`, `advantage_normalizer = "std"`, `importance_reweighting_method = "none"`, and `loss_normalization = "sequence"`. Feel free to raise a `NotImplementedError` for unsupported inputs.

The following interface is recommended:

```
def grpo_train_step(
    model: PreTrainedModel,
    tokenizer: PreTrainedTokenizer,
    optimizer: Optimizer,
    gradient_accumulation_steps: int,
    max_grad_norm: float | None,
    reward_fn: Callable[[str, str], dict[str, float]],
    repeated_prompts: list[str],
    rollout_responses: list[str],
    repeated_ground_truths: list[str],
    group_size: int,
    # Reward normalization
    baseline: Literal["mean", "none"] = "mean",
    advantage_eps: float = 1e-6,
    advantage_normalizer: Literal["std", "none", "mean"] = "std",
    # Importance reweighting and clipping
    importance_reweighting_method: Literal["none", "noclip", "grpo", "gspo"] = "none",
    old_log_probs: torch.Tensor | None = None,
    cliprange: float | None = None,
    # Loss normalization
    loss_normalization: Literal["sequence", "constant"] = "sequence",
    normalization_constant: int | None = None,
) -> tuple[torch.Tensor, dict[str, torch.Tensor | float]]:
```

Execute forward-and-backward passes, with `gradient_accumulation_steps` microbatches.

Args:

- **`model: PreTrainedModel`** HuggingFace model to train.
- **`tokenizer: PreTrainedTokenizer`** Tokenizer to use for tokenization.
- **`optimizer: Optimizer`** Optimizer for the model.
- **`gradient_accumulation_steps: int`** Number of microbatches per optimizer step.
- **`max_grad_norm: float | None`** If not `None`, clip the gradient norm to this value before calling `optimizer.step()`.
- **`reward_fn: Callable[[str, str], dict[str, float]]`** Scores the rollout responses against the ground truths, producing a dict with keys `"reward"`, `"format_reward"`, and `"answer_reward"`.
- **`repeated_prompts: list[str]`** The prompts for the examples. The length of this list is `rollout_batch_size`, because the prompt for each example is repeated `group_size` times.
- **`rollout_responses: list[str]`** Rollouts from the policy. The length of this list is `rollout_batch_size = n_prompts_per_rollout_batch * group_size`.
- **`repeated_ground_truths: list[str]`** The ground truths for the examples. The length of this list is `rollout_batch_size`, because the ground truth for each example is repeated `group_size` times.
- **`group_size: int`** Number of responses per question (group).
- **`baseline: Literal["mean", "none"]`** If `mean`, subtract the per-group mean reward; if `none`, do nothing.
- **`advantage_eps: float`** Small constant to avoid division by zero in normalization.
- **`advantage_normalizer: Literal["std", "none", "mean"]`** If `std`, divide by the per-group standard deviation; if `none`, do nothing; if `mean`, divide by the per-group mean reward.
- **`importance_reweighting_method: Literal["none", "noclip", "grpo", "gspo"]`** `"none"`: no importance reweighting; `"noclip"`: apply importance reweighting without clipping; `"grpo"`: do PPO/GRPO-style token-level reweighting and clipping; `"gspo"`: do GSPO-style sequence-level reweighting and clipping.
- **`old_log_probs: torch.Tensor | None`** Required unless `importance_reweighting_method = "none"`; shape `(batch_size, sequence_length)`.
- **`cliprange: float | None = None`** Clip parameter ε, required when `importance_reweighting_method` is `"grpo"` or `"gspo"`.
- **`loss_normalization: Literal["sequence", "constant"] = "sequence"`** `"sequence"`: average loss over each sequence, then average over sequences; `"constant"`: normalize total loss by a constant (fixed for all of training).
- **`normalization_constant: int | None = None`** The constant to divide total loss by; required if `loss_normalization = "constant"`.

Returns:

- **`tuple[torch.Tensor, dict[str, torch.Tensor]].`**
  - **loss** scalar tensor. The batch loss, adjusted for gradient accumulation. We return this so we can log it.
  - **metadata** Dict with metadata from the underlying loss call, gradient norm before clipping, and any other statistics you might want to log.

To test your code, implement [`adapters.run_grpo_train_step`] . Then run `uv run pytest -k test_grpo_train_step_standard_on_policy` and confirm it passes.

4.3 Experiments

We're now ready to construct the full GRPO training loop, which should:

- Load model and datasets
- Initialize logging (e.g. Wandb), vLLM server, optimizer
- Perform the RL training loop

We've provided some suggested hyperparameters below. If your training script is correct, you should see the validation reward increase over time when training with these hyperparameters, for most random seeds.

```
n_train_examples = 6400
n_val_examples = 1024
num_rollout_steps = 200
learning_rate = 1e-5
rollout_batch_size = train_batch_size = 256
group_size = 8
gradient_accumulation_steps = 32
sampling_temperature = 1.0
sampling_max_tokens = 512
max_grad_norm = 1.0
optimizer = torch.optim.AdamW(
    policy.parameters(), lr=learning_rate, betas=(0.9, 0.95), weight_decay=0.0
)
```

Note that `rollout_batch_size` and `train_batch_size` count responses, not prompts. So `rollout_batch_size = train_batch_size = 256` means `32` prompts with `8` rollouts each.

As a reasonable default, you can evaluate on the validation set once every 10 rollout batches. You should make sure to evaluate on at least 1024 validation examples, as CoT/RL evaluations can be noisy. In addition to logging the metrics described in the previous subsection, it's also useful to log rollouts for a qualitative sense of what the model is doing. A reasonable default is to log the training rollouts for the current batch of rollouts every 40 rollout batches. Feel free to use data/gsm8k/train.jsonl for the training examples and data/gsm8k/test.jsonl for validation examples.

Because the policy gradient estimator can have high variance and RL is self-reinforcing, RL training run trajectories tend to have very high variance. Therefore, once you have confidence that the script is correct, we will ask you to run your RL loop with 4 random seeds. It's okay if your script is not fully reproducible given a fixed seed; you can also just run the experiment 4 times. In your plots, please plot metrics with some indication of the variance across runs (i.e. don't just plot the mean). Some reasonable options include:

- Plotting the mean across the 4 runs, and also plotting the actual runs directly (for readability, it might help to place the actual runs on a separate plot, or use lower opacity, less line thickness, or dashed lines)
- Plotting the mean along with a shaded confidence interval: specifically, for each step, compute the sample mean and sample standard deviation. A reasonable interval is then $\mu \pm 1.96 * \frac{\sigma}{\sqrt{n}}$, where n is the number of seeds.

- Plotting the mean along with the min and max at each step.

Problem (`grpo_experiments_standard_on_policy`): Use GRPO to improve OLMo-2-0425-1B performance on GSM8K (2 B200 hrs) (10 points)

(a) Write a script that runs the GRPO training loop given a model name, prompt, training set file path, validation set file path, sampling hyperparameters, and training hyperparameters. At a high level, the script will involve first initializing the vLLM server, wandb logging, datasets, model, and optimizer. Then, the training loop involves repeating the following: sync weights with the vLLM server, produce training rollouts, take policy gradients on training batches, and periodically check performance on the validation set (while logging generations).

Deliverable: A script to run GRPO (standard, on-policy) on task GSM8K and model OLMo-2-0425-1B.

(b) Run the script for roughly 50 steps and confirm that you see validation rewards improving, along with sensible rollouts over time. Note: it is normal for the reward to start close to zero, and depending on the random seed it may take a few steps before we sample rollouts with non-zero rewards and the model starts improving.

Deliverable: Evidence that convinced you that your script is correct (e.g. validation rewards improving each step, reasonable-looking rollouts).

(c) Once you have confidence that the script is correct, run it using the hyperparameters provided above, with 4 random seeds, for OLMo-2-0425-1B on GSM8K. You should use the zero-shot `r1_zero` prompt. If the code is correct, you should see that validation rewards improve as the model is trained.

Log the following metrics, and produce plots of them over time, while noting the variance between runs (e.g. by plotting mean/std or min/max over runs).

- The loss.
- Gradient norm.
- Token entropy.
- Train rewards (total, format).
- Val rewards (total, format).
- Val average response length.
- Anything else you think could be useful for debugging.

Please also log the rollouts periodically and observe them. Do the responses improve as a result of training?

Deliverable: A few sentences of commentary and a plot for each metric, describing how it changes over training and how much variance there is between runs.

Deliverable: A few examples of the model rollouts before and after training.

Deliverable: A procedure that achieves a final validation accuracy of at least 25%, averaged across random seeds.

Now that we have a working script, we can start tuning hyperparameters and doing ablations. We'll start by tuning the learning rate, which is the hyperparameter that typically has the largest effect on training.

Problem (`grpo_learning_rate`): Tune the learning rate (4 B200 hrs) (3 points)

Starting from the suggested hyperparameters, perform a sweep over the learning rates (at least one learning rate less than the recommended default, and one learning rate greater than it), and report the final validation rewards (or note divergence if the optimizer diverges). Based on the amount of variance you observed in the previous part, you should decide how many random seeds you want to use for each training run. For the rest of the assignment, feel free to use your tuned learning rate instead of the suggested default, if you would like.

Deliverable: A plot of final validation reward versus learning rate, with a few sentences of commentary.

One other choice that influences RL dynamics is the prompt: because RL involves upweighting model rollouts, the prompt can shape which rollouts the model explores during training. We'll explore the effects of prompt choice below.

Problem (`grpo_prompt_ablation`): Prompt ablation (4 B200 hrs) (3 points)

Run the GRPO script with the `question_only` prompt and the `r1_zero_three_shot` prompt, with a few random seeds. Compared to the zero-shot `r1_zero` prompt results above, how do these prompts perform? Which prompts have the best average reward, and the lowest variance? Are there other systematic differences in the other logged metrics? Based on the variance between runs, how confident are you in your findings?

Deliverable: Commentary, supported by plots for referenced metrics.

5 RL algorithm variants

GRPO is a popular choice for language model RL, but its algorithmic choices are contested in a variety of papers. In this assignment, we'll learn some of the theoretical arguments behind these choices, and perform controlled experiments to see for ourselves which algorithms are best (at least for our model and dataset combination). This section will cover the on-policy setting, and the next section will cover off-policy algorithms.

5.1 Dr. GRPO

During the derivation of GRPO, you may have noticed a series of choices that meant that the GRPO policy gradient estimator no longer has the "correct" expectation $\nabla_\theta J_\theta$, where J_θ is the model's expected reward. These two choices were standard deviation advantage normalization and sequence length normalization. The first ablation we'll consider, argued for in the Dr. GRPO paper [Z. Liu et al., 2025], is to undo these choices: remove std normalization, and divide the total loss by a constant rather than first normalizing each sequence by its length. Denoting this constant normalizer as Z, the Dr. GRPO gradient estimator is then given by

$$\hat{g} \leftarrow \frac{1}{Z} \sum_{i=1}^{B} \sum_{j=1}^{G} \sum_{t=1}^{\text{len}(y^{(i,j)})} \left(r\left(y^{(i,j)} \mid x^{(i)}\right) - \mu_i\right) \nabla_\theta \log \pi_\theta\left(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)}\right), \tag{33}$$

where Z is typically set to the training batch size times the max generation length (i.e. $Z = BGL$, where L is the max generation length (512 in our case)).

Problem (`think_about_length_normalization`): Think about length normalization (1 point)

Before running any experiments, think about the difference between normalizing each sequence by sequence length, versus normalizing all sequences by the same constant. What are the pros and cons of each approach? Are there specific settings or examples where one approach seems better?

Deliverable: A few sentences of discussion.

Problem (`compute_group_normalized_rewards_drgrpo`): Dr. GRPO Group normalization (0.5 points)

Deliverable: Update your method `compute_group_normalized_rewards` to support `advantage_normalizer = "none"`. While Dr. GRPO uses `baseline = "mean"`, for this problem, please also add support for `baseline = "none"`.

To test your code, implement [`adapters.run_compute_group_normalized_rewards`] . Then, run the test with `uv run pytest -k compute_group_normalized_rewards_drgrpo` and make sure your implementation passes it.

Problem (`aggregate_loss_across_microbatch_constant`): Dr. GRPO loss aggregation (0.5 points)

Deliverable: Update your method `aggregate_loss_across_microbatch` to support `loss_normalization = "constant"`.

To test your code, implement [`adapters.run_aggregate_loss_across_microbatch`] . Then run `uv run pytest -k test_aggregate_loss_across_microbatch_constant` and ensure the test passes.

5.2 Rejection fine tuning

The next ablation we'll look into is a much simpler algorithm, typically referred to as "rejection fine tuning" (RFT) or "expert iteration" (EI). As the name suggests, the algorithm involves sampling a bunch of rollouts, keeping the ones that are correct, and then doing supervised fine tuning (i.e. minimizing next word prediction log loss) on these correct rollouts. Specifically, our RFT gradient is

$$\hat{g} \leftarrow \nabla_\theta \left[\frac{1}{Z} \sum_{i=1}^{B} \sum_{j=1}^{G} \mathbb{1}\left\{ r\left(y^{(i,j)} \mid x^{(i)}\right) = 1 \right\} \sum_{t=1}^{\text{len}\left(y^{(i,j)}\right)} \log \pi_\theta\left(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)}\right) \right], \tag{34}$$

where Z is a constant normalizer (like in Dr. GRPO), and $\mathbb{1}\left\{ r\left(y^{(i,j)} \mid x^{(i)}\right) = 1 \right\}$ is an indicator function which keeps only correct responses. In the following problem, we'll think about whether the RFT gradient is actually a policy gradient (i.e. whether it actually optimizes the expected reward J_θ), and how it relates to GRPO. As before, when implementing this as a PyTorch loss, return the negative objective so that minimizing the loss performs gradient ascent on the estimator.

Problem (`think_about_rft`): Think about RFT (2 points)

Recall that the RFT objective (with constant normalization) is given by

$$J_\theta = \frac{1}{Z}\sum_x \sum_{j=1}^{G} \mathbb{1}\{r(y^{(j)} \mid x) = 1\} \log \pi_\theta(y^{(j)} \mid x), \tag{35}$$

where x is the prompt, each $y^{(j)}$ denotes a response $y^{(j)} \overset{\text{iid}}{\sim} \pi_\theta(\cdot \mid x)$, G is the number of generations we sample from the policy, Z is our constant normalizer, and r is the reward function. The RFT gradient is then given by $\nabla_\theta J_\theta$.

In contrast, the on-policy Dr. GRPO policy gradient estimator (i.e. GRPO with constant normalization and no std normalization) is given by

$$\frac{1}{Z}\sum_x \sum_{j=1}^{G} (r(y^{(j)} \mid x) - \mu) \nabla_\theta \log \pi_\theta(y^{(j)} \mid x), \tag{36}$$

where μ denotes the group mean $\mu = \frac{1}{G}\sum_{j=1}^{G} r(y^{(j)} \mid x)$. Assuming our reward function r is binary, compare and contrast these objectives. Do they have the same expectation? Which do you expect to have lower variance? Intuitively, are there situations where you would prefer one or the other?

Deliverable: A few sentences of discussion.

5.3 MaxRL

Finally, we'll explore a recent method called Maximum Likelihood Reinforcement Learning, also known as MaxRL [F. Tajwar et al., 2026]. Instead of normalizing the advantage by the group std as in GRPO, or by nothing as in Dr. GRPO, MaxRL proposes to divide by the group mean. This choice results in the following gradient estimator:

$$\hat{g} \leftarrow \frac{1}{Z}\sum_{i=1}^{B}\sum_{j=1}^{G}\sum_{t=1}^{\text{len}(y^{(i,j)})} \frac{r(y^{(i,j)} \mid x^{(i)}) - \mu_i}{\mu_i} \nabla_\theta \log \pi_\theta\left(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)}\right). \tag{37}$$

Note that in the original MaxRL paper, instead of a constant normalizer Z, they divide by the total number of tokens in the batch $\sum_{i=1}^{B}\sum_{j=1}^{G} \text{len}(y^{(i,j)})$ (which is now batch-dependent instead of constant). In this assignment, we'll use the constant normalizer instead so that there are fewer moving parts relative to our other baselines, making it easier to isolate the effect of normalizing by μ_i instead of std_i.

In the following problem, we'll do some basic math to explore the effects of different advantage normalizers. Intuitively, dividing by average reward μ_i will on average upweight more difficult prompts, which are more likely to have lower μ_i compared to easier prompts. We can formalize this intuition by defining a reweighted version of our expected reward:

$$J_{\theta,w} = E_{x\sim\rho}\left[w(x, \text{stopgrad}(\pi_\theta)) E_{y\sim\pi_\theta} r(y \mid x)\right], \tag{38}$$

where $w(x, \text{stopgrad}(\pi_\theta))$ reweights prompts x, e.g. based on their difficulty $\eta(x) = E_{y\sim\pi_\theta} r(y \mid x)$ captured by the current policy's expected reward on x, and stopgrad denotes the fact that we don't take a gradient through the reweighting function. In the following problem, we'll derive the difficulty reweightings for each of the advantage normalizers we've seen so far.

Problem (`derive_difficulty_reweightings`): Derive difficulty reweightings induced by different advantage normalizers (6 points)

Recall that in aggregate over prompts, the standard policy gradient is given by

$$\nabla_\theta J_\theta = \nabla_\theta E_{x\sim\rho}\Big[E_{y\sim\pi_\theta} r(y \mid x)\Big], \tag{39}$$

where ρ is our prompt distribution over prompts x, π_θ denotes our policy, and $r(y \mid x)$ denotes whether y is a correct response to x. In this problem, we will derive the surrogate objectives optimized by our GRPO variants, where these surrogates take the form

$$\nabla_\theta J_{\theta,w} = \nabla_\theta E_{x\sim\rho}\Big[w(x, \text{stopgrad}(\pi_\theta)) E_{y\sim\pi_\theta} r(y \mid x)\Big] \tag{40}$$

for some reweighting over prompts w (where we do not differentiate through w).

(a) Recall that the Dr. GRPO policy gradient estimator is given by

$$E_{x\sim\rho}\left[\frac{1}{Z}\sum_{j=1}^{G}\big(r\big(y^{(j)} \mid x\big) - \mu\big)\nabla_\theta \log \pi_\theta\big(y^{(j)} \mid x\big)\right] \tag{41}$$

where μ denotes the group mean $\mu = \frac{1}{G}\sum_{j=1}^{G} r\big(y^{(j)} \mid x\big)$. Setting the constant normalizer $Z = G$ and taking group size $G \to \infty$, for what reweighting function w is this estimator equivalent to optimizing the surrogate objective $J_{\theta,w}$?

Deliverable: An expression in terms of a subset of the problem parameters, with a few sentences of justification.

(b) Assuming constant normalization, the GRPO estimator differs from Dr. GRPO in that it divides by the standard deviation:

$$E_{x\sim\rho}\left[\frac{1}{Z}\sum_{j=1}^{G}\frac{r\big(y^{(j)} \mid x\big) - \mu}{\text{std}}\nabla_\theta \log \pi_\theta\big(y^{(j)} \mid x\big)\right], \tag{42}$$

where std denotes the group standard deviation. Setting the constant normalizer $Z = G$ and taking group size $G \to \infty$, for what reweighting function w is this estimator equivalent to optimizing the surrogate objective $J_{\theta,w}$?

Deliverable: An expression in terms of a subset of the problem parameters, with a few sentences of justification.

(c) MaxRL instead divides by the group mean, or

$$E_{x\sim\rho}\left[\frac{1}{Z}\sum_{j=1}^{G}\frac{r\big(y^{(j)} \mid x\big) - \mu}{\mu}\nabla_\theta \log \pi_\theta\big(y^{(j)} \mid x\big)\right]. \tag{43}$$

Setting the constant normalizer $Z = G$ and taking group size $G \to \infty$, for what reweighting function w is this estimator equivalent to optimizing the surrogate objective $J_{\theta,w}$?

Deliverable: An expression in terms of a subset of the problem parameters, with a few sentences of justification.

Problem (`think_about_advantage_normalization`): Think about advantage normalization (2 points)

Before running any experiments, think about the difference between normalizing the group advantages by the group std, the group mean, or doing no advantage normalization. What are the pros and cons of each approach? Are there specific settings or examples where one approach seems better?

Deliverable: A few sentences of discussion.

Problem (`compute_group_normalized_rewards_maxrl`): MaxRL Group normalization (0.5 points)

Deliverable: Update your method `compute_group_normalized_rewards` to support `advantage_normalizer = "mean"`. Like `advantage_normalizer = "std"`, you should add `advantage_eps` to the normalizer to avoid division by zero.

To test your code, implement [`adapters.run_compute_group_normalized_rewards`] . Then, run the test with `uv run pytest -k compute_group_normalized_rewards_maxrl` and make sure your implementation passes it.

5.4 Experiments

We're now ready to run experiments to explore these different policy gradient estimators. First, we'll need to update our `grpo_train_step` method to support these variations, whose components we implemented above. The specific settings are as follows:

- `GRPO_constant`: `baseline = "mean"`, `advantage_normalizer = "std"`, `loss_normalization = "constant"`
- `Dr_GRPO`: `baseline = "mean"`, `advantage_normalizer = "none"`, `loss_normalization = "constant"`
- `RFT`: `baseline = "none"`, `advantage_normalizer = "none"`, `loss_normalization = "constant"`
- `MaxRL` (with constant normalization): `baseline = "mean"`, `advantage_normalizer = "mean"`, `loss_normalization = "constant"`

To speed up training, note that once we've computed normalized advantages for each sequence, sequences with zero advantage don't need to be passed into the model since their contribution to the gradient is zero. Since our rewards are binary, sequences will have zero advantage in two cases:

- If `baseline = "mean"`, the sequences in a group will have zero advantage if they all have the same reward.
- If `baseline = "none"`, any sequences with zero reward will have zero advantage.

Once we prune zero-advantage sequences, we can also reduce `gradient_accumulation_steps` by the same factor (i.e. we can keep our `microbatch_size` the same): for example, if half the sequences have zero advantage, we can take $\frac{k}{2}$ grad accum steps instead of k grad accum steps. This optimization can be especially helpful for `RFT`, which will have quite a few zero-reward sequences. But you should be careful with your math and implementation to ensure that your pruned version computes the same gradient as the unpruned version.

Problem (`grpo_train_step_variants_on_policy`): GRPO train step variants (2.5 points)

Deliverable: Update your `grpo_train_step` method to support the full range of on-policy variants (still with `importance_reweighting_method = "none"`). These options include `baseline: Literal["mean", "none"]`, `advantage_normalizer: Literal["std", "none", "mean"]`, and `loss_normalization: Literal["sequence", "constant"]`. Also, to speed up training, your method should avoid passing zero advantage sequences into the model.

Note that for `baseline = "none"`, the incorrect samples get zero reward and therefore zero weight in the loss, so it is unnecessary to pass them through the model. You can take advantage of this fact to adjust the implementation and speed up training.

To test your code, implement [`adapters.run_grpo_train_step`] . Then run `uv run pytest -k test_grpo_train_step_variants_on_policy` and confirm it passes.

Now that our training step is implemented, we can run our experiments to observe the performance of our methods, and decide whether they outperform standard GRPO. You should use the same hyperparameters as your previous runs to make your runs comparable.

A note on tuning hyperparameters: If our goal is to determine whether one method outperforms another, the most experimentally sound approach is to tune the hyperparameters separately for each method. However, hyperparameter tuning is expensive and we don't have that much compute for this assignment. The other, less expensive option is to tune hyperparameters for the baseline, and then use those hyperparameters for the new methods (e.g. in our case, we tuned learning rate on standard GRPO and are using that learning rate for the following experiments). In this case, if our new method outperforms the baseline, we can still claim that it is better, since the suboptimal learning rate produces a lower bound for the new method's properly tuned performance. But on the flip side, if the new method performs worse, we cannot claim it is worse than the baseline because we did not properly tune its hyperparameters.

Problem (`grpo_experiments_variants_on_policy`): Compare the performance of different RL algorithms (8 B200 hrs) (10 points)

Keeping hyperparameters fixed with respect to your standard GRPO training runs (for your choice of learning rate), run the following variations, with 4 random seeds each. You should use the zero-shot `r1_zero` prompt.

- `GRPO_constant`: standard GRPO, but doing constant normalization for loss aggregation, rather than sequence normalization.
- `Dr_GRPO`: `GRPO_constant` but with `advantage_normalizer = "none"`.
- `RFT`: `GRPO_constant` but with `advantage_normalizer = "none"` and `baseline = "none"`. Note that for this method, there is no need to pass incorrect samples through the model being trained, which should speed up training.
- `MaxRL`: `GRPO_constant` but with `advantage_normalizer = "mean"`. Note that the original MaxRL implementation normalizes the loss by the number of tokens per microbatch, but we will use constant normalization instead.

  Compared to the standard GRPO runs you ran previously, how do these methods perform? Which methods have the best average performance, and which methods have the lowest variance? Are there methods that you think would benefit from more hyperparameter tuning? Based on the variance between runs, how confident are you in your findings?

Deliverable: Commentary, supported by plots for referenced metrics.

6 Off-policy RL

Our experiments and algorithms so far have been *fully on-policy*, meaning that every gradient estimator used samples drawn directly from the model being updated (i.e. we took 1 training step per inference batch, and training batch size was set equal to inference batch size). In this section, we'll explore *off-policy RL*, which takes multiple training steps per inference batch and has the potential to speed up training, at the cost of potential instability and increased algorithmic complexity.

6.1 Importance reweighting

In the vanilla version of policy gradients, we sample a batch of responses, and then take a single large-batch gradient step on these responses. But we might hope for the model to train faster if we split the batch into a series of minibatches, and then take one training step per minibatch. This approach is called "off-policy RL", in contrast with our original approach of "on-policy RL" where we take one train step per inference batch.

The reason this approach is called off-policy RL is that after the first minibatch step, the current policy now differs from the policy we used during inference. The samples are now "off-policy" or "stale": they are not from the current policy, for which we would like to compute a policy gradient. In notation, if we denote the inference policy as π_0 and the current policy as π_θ, we have the following:

$$E_{x\sim\rho}E_{y\sim\pi_0(y\mid x)}[r(y\mid x)\nabla_\theta \log \pi_\theta(y\mid x)] \neq E_{x\sim\rho}E_{y\sim\pi_\theta(y\mid x)}[r(y\mid x)\nabla_\theta \log \pi_\theta(y\mid x)]. \tag{44}$$

The left-hand side is our "naive" off-policy estimator, where we pretend responses y came from our current policy and mechanically apply the same steps to compute our policy gradient. This equation says that this naive estimator does not have the correct expectation.

One way to correct this bias is to use *importance reweighting*, which produces the following estimator:

$$E_{x\sim\rho}E_{y\sim\pi_0(y\mid x)}\left[\frac{\pi_\theta(y\mid x)}{\pi_0(y\mid x)} r(y\mid x)\nabla_\theta \log \pi_\theta(y\mid x)\right], \tag{45}$$

where we've applied *importance weights* $\frac{\pi_\theta(y\mid x)}{\pi_0(y\mid x)}$. These weights have the effect of upweighting y's that are still relevant for the current policy π_θ, while downweighting "stale" samples with low $\pi_\theta(y\mid x)$. It is not too hard to see that this modified expression has the right expectation:

$$E_{x\sim\rho}E_{y\sim\pi_0(y\mid x)}\left[\frac{\pi_\theta(y\mid x)}{\pi_0(y\mid x)} r(y\mid x)\nabla_\theta \log \pi_\theta(y\mid x)\right] = E_{x\sim\rho}E_{y\sim\pi_\theta(y\mid x)}[r(y\mid x)\nabla_\theta \log \pi_\theta(y\mid x)]. \tag{46}$$

The downside is that importance reweighting increases the variance of our estimator: if the importance weight $\frac{\pi_\theta(y\mid x)}{\pi_0(y\mid x)}$ can be as large as C, our variance can also blow up by a factor of C. This problem makes naive importance reweighting inapplicable to language models, whose importance weight is a product over $\text{len}(y)$ terms, making it exponential in response length:

$$\frac{\pi_\theta(y\mid x)}{\pi_0(y\mid x)} = \prod_{t=1}^{\text{len}(y)} \frac{\pi_\theta(y_t\mid x, y_{<t})}{\pi_0(y_t\mid x, y_{<t})}. \tag{47}$$

So compared to the "naive" unweighted off-policy estimator, which has high bias but no additional variance introduced by reweighting, this reweighted estimator has zero bias but high variance. The following methods will lie somewhere in the middle, proposing various heuristics to navigate this bias-variance tradeoff.

6.2 PPO/GRPO-style importance reweighting and clipping

6.2.1 Token-level reweighting

The standard approach, first proposed in PPO [J. Schulman et al., 2017] and also adopted in GRPO [Z. Shao et al., 2024], is to do token-level importance reweighting instead of sequence-level reweighting. This choice results in the following "unclipped token-level reweighting" estimator:

$$E_{x\sim\rho}E_{y\sim\pi_0(y\mid x)}\left[r(y\mid x)\sum_{t=1}^{\text{len}(y)}\frac{\pi_\theta(y_t\mid x,y_{<t})}{\pi_0(y_t\mid x,y_{<t})}\nabla_\theta\log\pi_\theta(y_t\mid x,y_{<t})\right]. \tag{48}$$

Note that this expression is not the same as the "principled" sequence-level estimator we looked at above, which can be rewritten as the following (expanding over timesteps):

$$E_{x\sim\rho}E_{y\sim\pi_0(y\mid x)}\left[r(y\mid x)\left(\prod_{t=1}^{\text{len}(y)}\frac{\pi_\theta(y_t\mid x,y_{<t})}{\pi_0(y_t\mid x,y_{<t})}\right)\sum_{t=1}^{\text{len}(y)}\nabla_\theta\log\pi_\theta(y_t\mid x,y_{<t})\right]. \tag{49}$$

Before diving into what this modified expression is doing, note that the reweighting terms are no longer exponential in $\text{len}(y)$, drastically reducing variance.

Like previous sections of the assignment, to understand this estimator better, we'll derive the surrogate objective that this estimator is optimizing [T. Degris et al., 2013]. Specifically, let $\tilde{\pi}_t$ denote the surrogate policy where we sample all time steps from our old policy π_0, except step t which is sampled from π_θ:

$$\tilde{\pi}_t(y\mid x)=\left(\prod_{s=1}^{t-1}\pi_0(y_s\mid x,y_{<s})\right)\pi_\theta(y_t\mid x,y_{<t})\left(\prod_{s=t+1}^{\text{len}(y)}\pi_0(y_s\mid x,y_{<s})\right). \tag{50}$$

Then, if we assume responses all have the same length L, it turns out that token-level reweighting optimizes the expected reward under each $\tilde{\pi}_t$, or

$$J_\theta^{\text{token}}=E_x\left[\sum_{t=1}^{L}E_{y\sim\tilde{\pi}_t(y\mid x)}[r(y\mid x)]\right]. \tag{51}$$

This fact follows directly from rewriting this surrogate objective with importance reweighting terms:

$$\begin{aligned}\nabla_\theta J_\theta^{\text{token}}&=\nabla_\theta E_x\left[\sum_{t=1}^{L}E_{y\sim\tilde{\pi}_t(y\mid x)}[r(y\mid x)]\right] &(52)\\ &=\nabla_\theta E_x\left[\sum_{t=1}^{L}E_{y\sim\pi_0(y\mid x)}\left[\frac{\pi_\theta(y_t\mid x,y_{<t})}{\pi_0(y_t\mid x,y_{<t})}r(y\mid x)\right]\right] &(53)\\ &=E_x\left[E_{y\sim\pi_0(y\mid x)}\left[\sum_{t=1}^{L}\frac{\pi_\theta(y_t\mid x,y_{<t})}{\pi_0(y_t\mid x,y_{<t})}r(y\mid x)\nabla_\theta\log\pi_\theta(y_t\mid x,y_{<t})\right]\right], &(54)\end{aligned}$$

where the last step follows from the log derivative trick we've been using throughout this assignment.

This surrogate objective gives us an intuitive picture of the bias that token-level reweighting introduces. First, for the t-th term, the prefix $y_{<t}$ is drawn from π_0 rather than from π_θ, so it might not be representative of prefixes from the current model. Second, after we sample the token y_t from our current policy π_θ, the suffix $y_{>t}$ is drawn from π_0 rather than from π_θ. In other words, when the objective evaluates how the action y_t affects future steps, it does this evaluation based on the behavior of the stale policy π_0, rather than the current policy π_θ. Both of these biases are corrected by sequence-level

reweighting, which properly reweights prefixes and suffixes (at the cost of increased variance). Intuitively, these biases will be more severe as π_θ moves farther from π_0.

Problem (`derive_surrogate_objectives`): Derive surrogate objectives for importance reweighting approaches (2 points)

Above, we saw that token-level importance reweighting optimizes expected reward under surrogate policies where we sample from the old policy for all but one position. Suppose we instead consider a "pairwise" importance reweighting approach given by the following policy gradient estimator:

$$\sum_{t=1}^{\frac{L}{2}} \frac{\pi_\theta(y_{2t-1} \mid x, y_{<2t-1})\pi_\theta(y_{2t} \mid x, y_{<2t})}{\pi_0(y_{2t-1} \mid x, y_{<2t-1})\pi_0(y_{2t} \mid x, y_{<2t})} r(y \mid x)\nabla_\theta[\log(\pi_\theta(y_{2t-1} \mid x, y_{<2t-1})\pi_\theta(y_{2t} \mid x, y_{<2t}))] \quad (55)$$

where π_θ is our current policy, π_0 is our stale sampling policy, and $y \sim \pi_0(y \mid x)$. What surrogate objective is this estimator optimizing?

Deliverable: An expression in terms of a subset of the problem parameters, accompanied by a derivation.

6.2.2 Clipping

In addition to token-level reweighting, the other heuristic introduced by PPO and GRPO is importance weight clipping. As we saw above, the variance of importance reweighting becomes more severe as π_θ moves farther from π_0. Our surrogate objective tells us that the bias of token-level reweighting also becomes more severe. So to keep off-policy RL stable, we need to ensure π_θ stays close to π_0.

The simplest way to ensure π_θ stays close to π_0 is to reduce the number of training steps per inference batch, but some inference batches might allow for more steps. To squeeze performance out of our algorithm, we want to take as many steps as our inference batch allows us to. Toward this goal, one family of approaches involves clipping terms with large or small importance weight.

Recent papers disagree on the details of how clipping should be implemented, but in this assignment we'll implement the clipping approach from PPO, which has persisted to GRPO and seems to be the predominant approach. First, combining token-level reweighting with the standard GRPO objective produces the following loss function:

$$J_\theta^{\text{GRPO-off-policy-noclip}} = \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\frac{1}{\text{len}(y^{(i,j)})}\frac{r(y^{(i,j)} \mid x^{(i)}) - \mu_i}{\text{std}_i}\sum_{t=1}^{\text{len}(y^{(i,j)})}\frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_0(y_t \mid x, y_{<t})}, \quad (56)$$

where samples are drawn from our stale inference policy π_0. You can check that differentiating this objective gives us the token-reweighted policy gradient estimator we derived above, but with the GRPO advantages and sequence normalization.

We're ready to add clipping to this objective. Letting $A^{(i,j)} = \frac{r(y^{(i,j)} \mid x^{(i)}) - \mu_i}{\text{std}_i}$ denote the advantage of response j on prompt i, and letting $w_t^{(i,j)} = \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_0(y_t \mid x, y_{<t})}$ denote the t-th importance reweighting term, the clipped version of this objective is the following:

$$J_\theta^{\text{GRPO-off-policy-clip}} = \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\frac{1}{\text{len}(y^{(i,j)})}\sum_{t=1}^{\text{len}(y^{(i,j)})}\min\Big(A^{(i,j)}w_t^{(i,j)}, A^{(i,j)}\ \text{clip}\Big(w_t^{(i,j)}, [1-\varepsilon, 1+\varepsilon]\Big)\Big), \quad (57)$$

where $\text{clip}(w, [1-\varepsilon, 1+\varepsilon]) = \min(\max(w, 1-\varepsilon), 1+\varepsilon)$ clips the importance weight to the interval $[1-\varepsilon, 1+\varepsilon]$. A slightly more intuitive way to write this objective is the following [J. Achiam, 2018]:

$$J_\theta^{\text{GRPO-off-policy-clip}} = \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\frac{1}{\text{len}(y^{(i,j)})}\sum_{t=1}^{\text{len}(y^{(i,j)})}\begin{cases}\min\left(w_t^{(i,j)}, 1+\varepsilon\right)A^{(i,j)} \text{ if } A^{(i,j)} \geq 0 \\ \max\left(w_t^{(i,j)}, 1-\varepsilon\right)A^{(i,j)} \text{ if } A^{(i,j)} < 0\end{cases}. \tag{58}$$

Under this objective, the model is incentivized to upweight actions with high advantage, up until it weights that action $1+\varepsilon$ more than the old policy. At that point, the min takes the $1+\varepsilon$ path, and differentiating through it produces zero gradient. The negative advantage actions are similar: the model is incentivized to downweight the action until it weights that action $1-\varepsilon$ less than the old policy, at which point the gradient for that action becomes zero. So taking the gradient, we get the following policy gradient estimator:

$$\nabla_\theta J_\theta^{\text{GRPO-off-policy-clip}} = \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\frac{1}{\text{len}(y^{(i,j)})}\sum_{t=1}^{\text{len}(y^{(i,j)})}\text{mask}_t^{(i,j)} w_t^{(i,j)} A^{(i,j)} \nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t}) \tag{59}$$

with a mask function

$$\text{mask}_t^{(i,j)} = \begin{cases}\mathbb{1}\left\{w_t^{(i,j)} < 1+\varepsilon\right\} \text{ if } A^{(i,j)} \geq 0 \\ \mathbb{1}\left\{w_t^{(i,j)} > 1-\varepsilon\right\} \text{ if } A^{(i,j)} < 0\end{cases} \tag{60}$$

that masks out large importance weights for positive advantages, and masks out small importance weights for negative advantages.

We now have the math we need to implement `importance_reweighting_method = "noclip"` and `importance_reweighting_method = "grpo"` in our loss function, which should correspond to $J_\theta^{\text{GRPO-off-policy-noclip}}$ and $J_\theta^{\text{GRPO-off-policy-clip}}$. As in the on-policy case, return the negative objective from your loss implementation so that gradient descent performs the intended gradient ascent update.

Problem (`compute_policy_gradient_loss_off_policy`): Off-policy policy gradient with token-level reweighting (1 point)

Deliverable: Update your method `compute_policy_gradient_loss` to support `importance_reweighting_method = "noclip"` or `"grpo"`, which use the `old_log_probs` and `cliprange` arguments. `old_log_probs` denotes the per-token log probabilities under the model used to generate the rollouts, and you'll have to add a few lines to your training script to compute these. `cliprange` denotes the clipping strength parameter ε.

To test your code, implement [**`adapters.run_compute_policy_gradient_loss`**] . Then run `uv run pytest -k test_compute_policy_gradient_loss_off_policy` and ensure the test passes.

Note: while clipping is motivated in PPO as a way to prevent the current policy from straying too far from the old policy, it can also be viewed as a way to reduce the variance of the importance reweighted estimator, since we are squashing importance weight terms that are too large. If we are less concerned with straying far from the old policy and want to be more aggressive in our updates, a much simpler and more direct way to control the variance is to just upper-bound-clip the importance weight in the gradient estimator directly:

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^{B} \sum_{j=1}^{G} \frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \min\Big(w_t^{(i,j)}, 1+\varepsilon\Big) A^{(i,j)} \nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t}), \tag{61}$$

where unlike PPO/GRPO we still take non-zero gradient through actions that are upweighted by more than $1+\varepsilon$. This simpler clipping procedure is proposed in CISPO [MiniMax et al., 2025] (note: they also normalize by per-group token count instead of per-sequence length, but we write the sequence-normalized objective for simplicity). Optionally, you're welcome to try out CISPO and compare it to the other clipping methods.

6.3 GSPO

As we saw above, token-level reweighting is biased, in that it ignores the prefix and suffix reweighting. In the GSPO paper [C. Zheng et al., 2025], the authors find that GRPO is unstable and argue that we should do sequence-level reweighting instead. To get around the fact that the sequence-level importance weight is a product of L terms, they simply take the expression to the power $\frac{1}{L}$. In other words, they reweight by the geometric mean importance weight over the sequence, rather than the product. The result is the following loss function:

$$J_\theta^{\text{GRPO-off-policy-gspo}} = \frac{1}{BG} \sum_{i=1}^{B} \sum_{j=1}^{G} \min\big(A^{(i,j)} s^{(i,j)}, A^{(i,j)} \ \text{clip}\big(s^{(i,j)}, [1-\varepsilon, 1+\varepsilon]\big)\big), \tag{62}$$

where we now have a sequence-level importance weight $s^{(i,j)}$ that is shared across time steps:

$$s^{(i,j)} = \left(\prod_{t=1}^{\text{len}(y^{(i,j)})} \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_0(y_t \mid x, y_{<t})} \right)^{\frac{1}{\text{len}(y^{(i,j)})}}. \tag{63}$$

Note that without the power to the $\frac{1}{\text{len}(y^{(i,j)})}$ and without clipping, this objective becomes the sequence-level importance-reweighted policy gradient loss. Taking the geometric mean and clipping reduce variance at the cost of bias. One other note is that when taking the gradient, we naturally get sequence length normalization due to the geometric mean power (clipping is ignored in the equation below for brevity):

$$\nabla_\theta J_\theta^{\text{GRPO-off-policy-gspo}} = \frac{1}{BG} \sum_{i=1}^{B} \sum_{j=1}^{G} A^{(i,j)} \nabla_\theta s^{(i,j)} \tag{64}$$

$$= \frac{1}{BG} \sum_{i=1}^{B} \sum_{j=1}^{G} A^{(i,j)} s^{(i,j)} \frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t})). \tag{65}$$

So if you want to apply GSPO to your favorite constant-normalized RL algorithm (optional), you will need to change the exponent in your sequence-level importance weight.

Problem (`think_about_importance_reweighting`): Think about importance reweighting (2 points)

Consider three possible importance reweighting strategies for off-policy RL: (a) no importance reweighting, (b) PPO/GRPO-style clipped token-level importance reweighting, and (c) GSPO-style clipped sequence-level importance reweighting using the geometric mean. In terms of trading off bias and variance, where does each approach lie on the spectrum? Can you think of situations where one approach might intuitively be better than the others?

Deliverable: A few sentences of commentary.

In the following problem, we will implement the GSPO loss. Remember to compute the geometric mean in a numerically stable way.

Problem (`compute_policy_gradient_loss_off_policy_gspo`): Off-policy policy gradient with sequence-level reweighting (1 point)

Deliverable: Update your method `compute_policy_gradient_loss` to support `importance_reweighting_method = "gspo"`, which uses the `old_log_probs` and `cliprange` arguments. `old_log_probs` denotes the per-token log probabilities under the model used to generate the rollouts, and you'll have to add a few lines to your training script to compute these. `cliprange` denotes the clipping strength parameter ε. As above, return the negative objective so that minimizing the loss performs gradient ascent.

To test your code, implement [`adapters.run_compute_policy_gradient_loss`] . Then run `uv run pytest -k test_compute_policy_gradient_loss_off_policy_gspo` and ensure the test passes.

6.4 Experiments

We now have the components necessary to update our `grpo_train_step` method to support off-policy training.

Problem (`grpo_train_step_off_policy`): Off-policy GRPO train step (2.5 points)

Deliverable: Update your `grpo_train_step` method to support the off-policy arguments (`importance_reweighting_method`, `old_log_probs`, `cliprange`). The function should now support the full range of arguments.

To test your code, implement [`adapters.run_grpo_train_step`] . Then run `uv run pytest -k test_grpo_train_step_off_policy` and confirm it passes.

Does making training off-policy actually make it faster, and to what extent do we lose stability? And which reweighting/clipping method is best? We'll explore these questions in the experiments below. Please keep hyperparameters fixed with respect to your previous RL runs, except the runs should now be 32x off policy (so we take 32 training steps for each inference batch):

```
rollout_batch_size = 256
train_batch_size = 8
gradient_accumulation_steps = 1
```

Also, as a default, you can use the following clipping range values (i.e. ε in the derivations above), taken from the original papers. Optionally, you're also welcome to tune these values yourself.

- `offpolicy_clip`: `cliprange = 0.2`
- `offpolicy_gspo`: `cliprange = 3e-4`

Note that you'll also have to add a few lines to your training script to compute `old_log_probs`, to pass into your reweighted loss.

Problem (`grpo_experiments_off_policy`): Compare the performance of different off-policy algorithms (8 B200 hrs) (10 points)

Keeping hyperparameters fixed with respect to your standard GRPO training runs (for your choice of learning rate), run the following variations, with 4 random seeds each. You should use the zero-shot `r1_zero` prompt. If you would like, you can choose your favorite RL algorithm variant from the previous section instead of sticking with the standard version.

- `offpolicy_naive`: Instead of making inference and training batch size equal (both 256), reduce the training batch size to 1/32nd of the inference batch size ($\frac{256}{32} = 8$). Remember to reduce the number of gradient accumulation steps by the same factor so we continue fully utilizing our GPUs. Keep `importance_reweighting_method = "none"`.
- `offpolicy_noclip`: `offpolicy_naive` with `importance_reweighting_method = "noclip"`.
- `offpolicy_clip`: `offpolicy_naive` with `importance_reweighting_method = "grpo"`.
- `offpolicy_gspo`: `offpolicy_naive` with `importance_reweighting_method = "gspo"`.

In addition to the metrics you're already logging, make sure to also log the clip fraction.

Compared to the fully on-policy GRPO runs you ran previously, how do these methods perform? How does going off-policy affect training stability and variance between runs? How do the two clipping methods differ in terms of the clip fraction, and which one seems more stable? Are there methods that you think would benefit from more hyperparameter tuning? Based on the variance between runs, how confident are you in your findings?

Deliverable: Commentary, supported by plots for referenced metrics.

7 Try your own policy gradient estimator

Problem (`try_your_own`): Try your own policy gradient estimator (10 points)

Now that you've seen a variety of policy gradient estimators that have been proposed in the literature, along with some of the theory behind them, it's time for you to propose your own! Here are some suggestions:

- You could try a different advantage estimator, inducing a different reweighting over prompts.
- You could try a different importance reweighting strategy.
- You could try a different reward baseline in the advantage estimator.

Propose your own policy gradient estimator. Then, run it on the same RL setting of OLMo-2-0425-1B on GSM8K and compare it to the approaches we've already tried as baselines, using multiple random seeds and keeping problem parameters fixed to make the results comparable. Please change just one thing relative to the approaches tried so far (i.e. there should be at least one previous approach covered in this assignment such that your approach changes only one thing).

What is the theoretical justification or intuition behind your proposed change? Does your proposed change beat the baselines?

Deliverable: Commentary, supported by plots for referenced metrics, and optionally mathematical derivations if they are useful in justifying your approach.

Bibliography

[1] J. Li *et al.*, "DataComp-LM: In search of the next generation of training sets for language models." [Online]. Available: https://arxiv.org/abs/2406.11794

[2] T. OLMo *et al.*, "2 OLMo 2 Furious." [Online]. Available: https://arxiv.org/abs/2501.00656

[3] K. Cobbe *et al.*, "Training Verifiers to Solve Math Word Problems." [Online]. Available: https://arxiv.org/abs/2110.14168

[4] DeepSeek-AI *et al.*, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning." [Online]. Available: https://arxiv.org/abs/2501.12948

[5] W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention." 2023.

[6] Z. Liu *et al.*, "Understanding R1-Zero-Like Training: A Critical Perspective." [Online]. Available: https://arxiv.org/abs/2503.20783

[7] OpenAI *et al.*, "OpenAI o1 System Card." [Online]. Available: https://arxiv.org/abs/2412.16720

[8] Z. Shao *et al.*, "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models." [Online]. Available: https://arxiv.org/abs/2402.03300

[9] J. Achiam, "Spinning Up in Deep Reinforcement Learning," 2018.

[10] N. Lambert, "Reinforcement Learning from Human Feedback." [Online]. Available: https://rlhfbook.com/

[11] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, 1992, doi: 10.1007/BF00992696.

[12] F. Tajwar *et al.*, "Maximum Likelihood Reinforcement Learning." [Online]. Available: https://arxiv.org/abs/2602.02710

[13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms." [Online]. Available: https://arxiv.org/abs/1707.06347

[14] T. Degris, M. White, and R. S. Sutton, "Off-Policy Actor-Critic." [Online]. Available: https://arxiv.org/abs/1205.4839

[15] J. Achiam, "Simplified PPO-Clip Objective." [Online]. Available: https://drive.google.com/file/d/1PDzn9RPvaXjJFZkGeapMHbHGiWWW20Ey/view

[16] MiniMax *et al.*, "MiniMax-M1: Scaling Test-Time Compute Efficiently with Lightning Attention." [Online]. Available: https://arxiv.org/abs/2506.13585

[17] C. Zheng *et al.*, "Group Sequence Policy Optimization." [Online]. Available: https://arxiv.org/abs/2507.18071